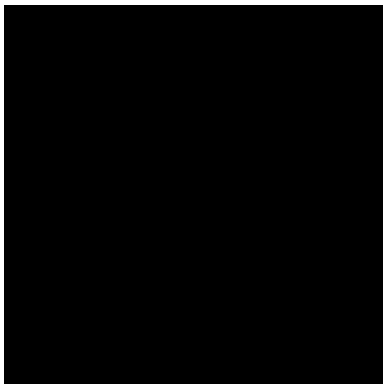
Port Said University
Faculty of Engineering
Department of Computer and Control Engineering

Kemora: An AI-Powered Travel Discovery Platform for Egypt

Graduation Project 2026



Supervisor	Dr. Rabab Farouk
Team Members	Mohamed Ahmed Mousad Mohamed Khaled Elsayed Zeyad Khaled Elsayed Mohamed Eltabey Elshenawy Ahmed Bassem Elshafey
Department Name	Department of Computer and Control Engineering
Date of Submission	July 2026

Abstract / Executive Summary

Problem Statement: The tourism industry in Egypt is heavily fragmented, leaving modern travelers struggling to plan logical itineraries while simultaneously discovering off-the-beaten-path "hidden gems." Existing tools either offer generic AI generation with severe geographic hallucinations or simple static maps with no intelligent routing.

Methodology: This project presents **Kemora**, a comprehensive TravelTech ecosystem. The solution integrates a high-performance .NET 10 Clean Architecture backend with a 60fps Flutter cross-platform mobile application. To solve AI hallucinations, Kemora implements a deterministic Retrieval-Augmented Generation (RAG) pipeline, dynamically fetching real, localized Points of Interest (POIs) from the Foursquare API and a seeded local database within a 30km radius before injecting them into an OpenRouter-hosted Large Language Model (LLM).

Key Findings: Empirical benchmarking confirms the backend successfully achieves median latencies of 22ms and sustains robust throughput under load via aggressive `IMemoryCache` permutations. The transition from sequential API fetching to in-memory Zero-Fetch Image Injection eradicated database concurrency bottlenecks.

Recommendations: Future iterations should transition to a Microservices Architecture deployed via Kubernetes, implement an offline-first mobile architecture utilizing SQLite/Hive, and natively fine-tune open-source LLMs using LoRA techniques to completely eliminate third-party dependencies.

Acknowledgments

The successful completion of our graduation project would not have been possible without the invaluable support and contributions of several individuals and groups.

First and foremost, we would like to express our sincere gratitude to our esteemed supervisor, Dr. Rabab Farouk. Her guidance and mentorship have been instrumental throughout this project. Her deep knowledge of the field and her ability to provide insightful feedback were crucial in shaping the direction of our research and ensuring its academic rigor. Her encouragement and unwavering belief in our abilities helped us navigate challenges and persevere during difficult moments.

We would also like to extend our heartfelt appreciation to our families for their unwavering support throughout this entire journey. Their patience and understanding during countless hours of research and writing were invaluable. Their words of encouragement during moments of doubt and their celebration of milestones kept us motivated and focused on achieving our goals.

In closing, we would like to express our gratitude to all those who have played a role in bringing this project to fruition.

Thank you all.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Tourism has historically been, and remains, a vital and foundational pillar of the Egyptian economy. Egypt boasts a uniquely diverse portfolio of attractions that spans millennia, ranging from the ancient Pharaonic monuments in Luxor and Aswan, to pristine coastal resorts and diving havens in the Red Sea and Sinai, alongside a burgeoning sector of eco-tourism and medical tourism. Recent statistics underscore the sector's monumental economic impact. During the 2023 and 2024 periods, the Egyptian tourism sector experienced record-breaking growth. In the 2024/2025 fiscal year, tourism revenues reached an unprecedented $16.7 billion, representing a massive 56.1% growth over previous periods, driven by an influx of over 15.7 million tourists. Furthermore, the sector's direct contribution to Egypt's GDP rose to 3.7%, with tourist nights increasing to approximately 179.3 million.

Despite this wealth of attractions and strong economic performance, the global travel industry is undergoing a rapid digital transformation, and the expectations of the modern traveler have evolved significantly. Today's tourists demand highly personalized, instantly accessible, and community-driven digital experiences. The academic literature on artificial intelligence (AI) in tourism frequently highlights *fragmentation* as a primary challenge that hinders the development of seamless, holistic, and user-centric travel experiences.

In the Egyptian context, travelers frequently encounter a highly fragmented digital ecosystem. To plan a comprehensive multi-day trip, a user must typically navigate multiple disparate platforms: one aggregator for flight and hotel bookings, a separate directory for discovering local restaurants or cafes, a third application for mapping and geographical logistics, and yet another social network for reading reviews or connecting with other travelers. This information fragmentation leads to cognitive overload and suboptimal travel planning. Furthermore, while generic global travel platforms exist, they often lack

the deep, localized context required to truly optimize an Egyptian itinerary. Such optimization requires understanding local transportation nuances, seasonal operating hours, and accurately grouping proximate attractions to minimize travel time within congested cities like Cairo or along the expansive Nile Valley.

The motivation for this graduation project stems from the urgent desire to unify these disjointed experiences into a single, cohesive, and intelligent platform. By leveraging the latest advancements in Artificial Intelligence—specifically Generative Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) paradigms—alongside modern cross-platform mobile development architectures, there is an unprecedented opportunity to automate the labor-intensive process of itinerary creation while simultaneously fostering a vibrant, localized community of travelers.

1.2 The Economic and Systemic Problem Statement

The core problem addressed by this project is the profound inefficiency and fragmentation of the modern travel planning and discovery process in Egypt. This fragmentation is not merely a user-experience hurdle; it represents a tangible economic problem. When travelers face friction in planning, they naturally default to the path of least resistance: visiting only the primary, globally recognized tourist hubs such as the Pyramids of Giza or the Luxor Temple. Consequently, thousands of "hidden gems"—ranging from authentic local eateries in Cairo's Islamic districts to pristine, lesser-known coastal eco-lodges along the Red Sea—remain undiscovered by the average tourist. This leads to an unequal distribution of tourism revenue, overcrowded primary landmarks, and a failure to showcase the true, multifaceted cultural depth of Egypt to the world. Kemora directly addresses this by making the discovery of these hidden gems as effortless as booking a ticket to the Pyramids.

Academic research indicates that existing digital solutions often specialize in isolated functionality rather than providing an end-to-end planning ecosystem. Specifically, the problems can be categorized into four primary domains:

1. **Complex and Time-Consuming Itinerary Generation:** Planning a multi-day trip requires solving what is essentially a variation of the Traveling Salesman Problem, balancing geographical proximity of activities, realistic time allocation,

fatigue management, and alignment with personal budgets and interests. Doing this manually across disparate apps is tedious and prone to human error.

2. **Limitations of Current AI and Propensity for Hallucination:** While Generative AI has democratized access to travel advice, generic AI chatbots (like standard ChatGPT without web-grounding) suffer from a lack of contextual awareness and severe hallucinations. In the tourism domain, this manifests as recommending non-existent places, permanently closed restaurants, or geographically impossible routes (e.g., scheduling a morning visit to the Pyramids of Giza and an afternoon lunch in Sharm El-Sheikh on the same day). The literature points to a significant data integration gap, where AI fails to utilize real-time, verified POI (Point of Interest) data.

3. **Disconnect Between Discovery and Community:** Existing platforms isolate the planning phase from the social sharing phase. Travelers rely on social media (e.g., Instagram, TikTok) for inspiration, but cannot easily convert a friend's travel post or a viral video directly into an actionable, mapped item on their own itinerary. This disconnect breaks the user journey.

4. **Low Engagement and Retention in Travel Apps:** Traditional utility-focused travel apps suffer from high churn rates; they are frequently deleted post-trip due to a lack of ongoing engagement mechanisms. Without a social or gamified component, there is no incentive for the user to return to the application until their next major vacation.

1.3 Competitive Analysis

To contextualize the proposed solution, it is necessary to conduct a deep competitive analysis comparing the envisioned system against existing market incumbents. The current landscape is dominated by four archetypes of platforms, each exhibiting critical shortcomings when applied to the Egyptian market:

1.3.1 Global Aggregators and Review Platforms (e.g., TripAdvisor, Expedia)

Strengths: Massive global databases of reviews, standardized booking interfaces, and high brand trust.

Weaknesses: These platforms are inherently broad and lack deep, localized curation. Their itinerary generation features are typically rigid, relying on pre-packaged static tours rather than dynamically generated, personalized schedules. Furthermore, they function primarily as utility directories rather than cohesive social networks, making organic discovery feel clinical rather than inspiring.

1.3.2 Generic AI Chatbots and Planners (e.g., ChatGPT, Roamr)

Strengths: Highly conversational, capable of understanding complex natural language constraints, and instantaneous generation of text-based itineraries.

Weaknesses: As highlighted in recent literature, these systems suffer from systemic data hallucinations. Because they are not deeply integrated with local Egyptian geographical APIs or live business status directories, they frequently recommend logically flawed routes or closed venues. They also lack a visual, interactive map-based interface to execute the plan.

1.3.3 Social Media Networks (e.g., Instagram, TikTok)

Strengths: Unparalleled visual inspiration, highly engaging algorithms, and massive user bases driving organic discovery of hidden gems.

Weaknesses: They are completely devoid of practical planning tools. When a user discovers a beautiful cafe in Alexandria on TikTok, transferring that inspiration into a structured, routed daily itinerary requires switching to Google Maps and manually saving coordinates. The inspiration cannot be natively converted into an actionable trip.

1.3.4 Localized Egyptian Directories (e.g., Elmenus, Cairo360)

Strengths: Excellent, highly accurate, and hyper-localized data regarding specific niches (e.g., Elmenus for dining, Cairo360 for events).

Weaknesses: They are heavily siloed. A traveler cannot use Elmenus to plan a visit to the Egyptian Museum, nor can they use it to book a hotel. They solve a micro-problem rather than the macro-problem of holistic travel planning.

1.4 Proposed Solution: Kemora

To overcome the limitations of the current fragmented landscape, we propose **Kemora**, an intelligent, all-in-one travel discovery and social platform natively designed for the Egyptian context. Kemora is envisioned as the ultimate companion for navigating Egypt, bridging the gap between AI-driven planning and human-centric social discovery. The platform's architecture provides four core pillars:

1.4.1 Intelligent Trip Planning via RAG

Kemora features a sophisticated AI-driven trip planner. By inquiring about the user's destination, budget, travel style (e.g., historical, relaxation, adventure), and duration, the system dynamically curates a day-by-day, hour-by-hour itinerary. Crucially, the system mitigates the AI hallucination problem by implementing a robust Retrieval-Augmented Generation (RAG)-like pipeline. Rather than relying on the LLM's parametric memory, the backend pre-fetches verified, highly-rated places from external APIs (such as Foursquare) within a strict geographical radius. This constrained, localized dataset is then injected into the LLM's context window. This strict architectural decision ensures the AI only recommends real, currently open, and geographically proximate locations, entirely eliminating impossible routing.

1.4.2 Social Discovery and Community Integration

Unlike utility-only apps, Kemora integrates a fully-fledged social network. Users can share posts, upload media, react, comment, and follow other travelers. This transforms the application from a transient planning tool into a source of continuous, daily inspiration. Most importantly, users can explore feeds of content generated by others and seamlessly "bookmark" or extract places mentioned in posts, directly injecting them into their personal itineraries. This solves the disconnect between inspiration and execution.

1.4.3 Interactive Exploration and Editorial Context

The application provides an interactive map and a rich directory of Egyptian governorates. Users can explore specific cities, read detailed editorial descriptions, view high-quality image galleries, and filter attractions by granular categories (Hotels, Restaurants, Cafes, Landmarks, Medical Tourism hubs). This ensures that even users who prefer manual planning over AI generation have access to a centralized, beautiful discovery interface.

1.4.4 Gamification and Retention Mechanisms

To drive continuous user engagement and combat post-trip app deletion, Kemora employs a deep gamification system. Users earn points, level up, and unlock badges for contributing to the community (e.g., publishing high-quality posts, leaving detailed reviews, completing planned trips). This system incentivizes a rich, user-generated content ecosystem that continually improves the platform's proprietary data quality and creates a sticky, retaining user experience.

1.5 Project Objectives

The primary objectives of this graduation project are to deliver a production-ready software ecosystem. Specifically:

- **Full-Stack Engineering:** To design, develop, and deploy a robust, highly responsive full-stack application utilizing a modern Flutter frontend for cross-platform mobile delivery, backed by a high-performance .NET 10 Web API.
- **Clean Architecture Implementation:** To rigorously apply Clean Architecture principles and Domain-Driven Design (DDD) on the backend. This ensures the system remains maintainable, inherently testable, and loosely coupled, allowing for future expansion.
- **AI Orchestration and Prompt Engineering:** To engineer a reliable AI pipeline that interacts with state-of-the-art models via OpenRouter. This involves dynamic

model selection, context window management, precise prompt engineering, and strict JSON schema enforcement to ensure parsable itinerary generation.

- **High Performance and Scalability:** To implement effective optimization strategies, including aggressive caching (`IMemoryCache`, distributed caching) and concurrent asynchronous data fetching, ensuring low-latency API responses even during complex AI generation tasks.

- **Quality Assurance and Benchmarking:** To rigorously validate the system's reliability through comprehensive End-to-End (E2E) integration testing and load benchmarking, proving the backend can withstand concurrent user traffic.

1.6 Document Organization

The remainder of this thesis is structured as follows:

- **Chapter 2 (Literature Review & Technologies):** Explores the theoretical foundations of the selected technology stack, including the evolution of Flutter, the performance characteristics of .NET, the principles of Clean Architecture, and the applied theory behind Large Language Models and RAG systems.

- **Chapter 3 (System Analysis & Design):** Details the system's non-functional and functional requirements, comprehensive UML diagrams (Use Case, Sequence, Activity), high-level system architecture, and complex database entity-relationship (ER) models.

- **Chapter 4 (Implementation):** Chronologically outlines the development phases, highlighting the evolution of the AI pipeline, UI/UX implementation challenges, and complex backend integrations such as authentication and external API routing.

- **Chapter 5 (Testing & Benchmarking):** Discusses the E2E testing methodology, unit testing coverage, and presents empirical results from backend load testing and performance profiling.

- **Chapter 6 (Conclusion & Future Work):** Summarizes the project's achievements, reflects on lessons learned, and outlines potential avenues for future commercial enhancement and feature expansion.

- **Appendix A:** Provides a comprehensive API documentation reference (Swagger/OpenAPI specifications) for the backend services.

Chapter 2

Literature Review & Technology Stack

2.1 Introduction

Building a modern, highly scalable, and artificial intelligence-integrated travel platform necessitates a meticulous and rigorous selection of underlying technologies. The architectural paradigm must seamlessly support real-time social interactions, highly complex relational data models, asynchronous integrations with external application programming interfaces (APIs), and deterministic, cross-platform mobile rendering at high frame rates. This chapter provides a comprehensive review of the theoretical background, mathematical foundations, and software engineering paradigms of the chosen technologies, establishing a scholarly justification for their selection over contemporaneous alternatives.

2.2 Mobile Frontend Development: Flutter and Impeller

2.2.1 Overview of the Flutter Framework

Flutter is an open-source, declarative UI software development kit (SDK) engineered by Google. Since its inception, it has rapidly ascended as the dominant framework for cross-platform mobile application development. Unlike traditional cross-platform frameworks such as Apache Cordova or Ionic, which rely on a WebView and HTML/CSS-based rendering, or frameworks like React Native, which utilize a JavaScript bridge to asynchronously invoke OEM (Original Equipment Manufacturer) widgets, Flutter adopts a radically divergent architectural approach. It ships with its own high-performance rendering engine, fundamentally bypassing OEM widgets to draw graphics primitives directly to the canvas provided by the host operating system.

2.2.2 The Impeller Rendering Engine

Historically, Flutter utilized the Skia 2D graphics library. However, Skia's reliance on Just-In-Time (JIT) shader compilation often led to "shader compilation jank"—dropped frames during the initial rendering of complex animations as the engine compiled shaders on the fly.

To resolve this bottleneck, Flutter transitioned to **Impeller**, a modern rendering engine built from the ground up for Flutter's specific needs. Impeller fundamentally alters the rendering pipeline through an **Ahead-of-Time (AOT)** approach.

- **AOT Shader Compilation:** Impeller pre-compiles a specialized, deterministic set of shaders at engine-build time. All pipeline state objects (PSOs) are generated upfront, mathematically guaranteeing that shader compilation will not occur during the rendering of an animation frame.
- **Modern Graphics APIs:** Impeller interfaces directly with low-level, modern graphics APIs—specifically **Metal** on iOS and **Vulkan** on Android (API 29+). This bypasses the overhead of legacy abstraction layers like OpenGL.
- **Tessellation-based Rendering:** For complex vector paths, Impeller utilizes tessellation algorithms to break curves into primitive triangles, heavily parallelizing the rasterization process across the GPU's multi-core architecture.

This architectural shift provides predictable, 120-frames-per-second (fps) performance, crucial for maintaining user immersion in a visually rich travel application like Kemora.

2.2.3 The Dart Programming Language

Flutter's underlying logic is executed via Dart, a client-optimized, strictly-typed, object-oriented language. Dart provides a dual-compilation paradigm that accelerates both the development lifecycle and runtime execution:

1. **Just-In-Time (JIT) Compilation:** During the development phase, the Dart Virtual Machine executes code via JIT compilation. This enables "Stateful Hot Reload," allowing developers to inject updated source code into the running virtual machine without losing the application state.

2. **Ahead-Of-Time (AOT) Compilation:** For production artifacts, Dart compiles directly to native ARM machine code. This eliminates the overhead of a virtual machine interpretation at runtime, ensuring deterministic memory allocation and fast startup times.

2.2.4 Declarative UI and State Management Architecture

Flutter employs a declarative UI paradigm, an evolution from imperative UI models where the developer manually mutates the UI state. In Flutter, the UI is a pure function of the state: $UI = f(state)$. Everything from padding models to interactive buttons is an immutable `Widget`. When the state changes, the framework reconstructs the widget tree and computes a diff against the element tree, selectively updating the underlying render objects.

For state management, Kemora integrates the `Provider` package, an optimized wrapper around Flutter's inherently $O(1)$ `InheritedWidget` propagation mechanism. This completely eliminates "prop-drilling" in deep widget trees. To maintain separation of concerns, the frontend architecture rigorously adheres to the principles of Clean Architecture, isolating Presentation logic (View and ViewModels) from Domain operations (Entities and UseCases).

2.3 Backend Architecture: .NET 10 and C#

2.3.1 Runtime Enhancements in .NET 10

The backend application programming interface (API) is powered by Microsoft's .NET 10, the most advanced iteration of the cross-platform .NET ecosystem. .NET 10 introduces critical enhancements at the runtime level. The Tiered Compilation system has been heavily optimized, incorporating aggressive loop unrolling, enhanced escape analysis, and refined Dynamic PGO (Profile-Guided Optimization). These JIT improvements allow the runtime to minimize heap allocations by promoting short-lived objects to the stack, significantly reducing the pressure on the Garbage Collector (GC) and enabling massive throughput under high concurrency.

2.3.2 Kestrel Web Server and TechEmpower Benchmarks

ASP.NET Core utilizes Kestrel, an asynchronous, event-driven web server. Kestrel's architecture relies heavily on non-blocking I/O multiplexing (e.g., `epoll` on Linux) and advanced memory pooling techniques via `Span<T>` and `Memory<T>`, enabling zero-copy parsing of HTTP requests.

Consequently, Kestrel consistently ranks in the top tier of the rigorous **TechEmpower Framework Benchmarks**. Specifically, in Plaintext and JSON serialization workloads, Kestrel demonstrates industry-leading throughput, capable of processing millions of requests per second on standard hardware. This raw performance is indispensable for Kemora, as the system orchestrates multiplexed asynchronous requests to external AI providers and geocoding services simultaneously.

2.4 Software Engineering Paradigms: Clean Architecture

2.4.1 The Dependency Rule and Inversion of Control

Clean Architecture, formalized by Robert C. Martin, dictates a ringed topological structure of software dependencies. The foundational axiom is the **Dependency Rule**: source code dependencies must exclusively point inward toward higher-level domain policies.

To achieve this without violating runtime execution requirements, the architecture leverages the **Dependency Inversion Principle (DIP)**, a cornerstone of SOLID design. Rather than the inner layers depending on external infrastructure (e.g., a database), the inner layer defines an abstraction (an `interface`), and the outer layer provides the concrete implementation. A robust **Dependency Injection (DI)** container resolves these interfaces at runtime.

2.4.2 Service Layer Architecture and the Repository Pattern

To manage the complexity of Kemora's data access and business rules, the application employs a **Service Layer** architecture. Rather than segregating reads and writes via a CQRS pattern, the conceptual model aggregates related domain behaviors into highly co-

hesive, modular service classes (e.g., `TripPlannerService`, `PlaceManagementService`). This design minimizes architectural boilerplate, encapsulates transaction boundaries, and coordinates workflows seamlessly between the web presentation layer and the domain logic.

Complementing the Service Layer is the **Repository Pattern**. The Repository acts as an in-memory collection abstraction over the persistent data store. By utilizing interface-driven repositories (e.g., `ITripRepository`), the domain logic remains entirely agnostic of the underlying ORM (Entity Framework Core) or relational database dialect.

2.5 The Mathematics of Large Language Models (LLMs)

2.5.1 Transformer Architecture and Token Processing

The generative AI capabilities within Kemora are powered by transformer-based Large Language Models (LLMs). Unlike legacy Recurrent Neural Networks (RNNs) that process sequences sequentially, the Transformer architecture introduced by Vaswani et al. (2017) processes tokens in parallel via the mechanism of Self-Attention.

The text processing pipeline involves several mathematical transformations:

1. **Tokenization and Embedding:** The input string is tokenized into a sequence of indices, $X = [x_1, x_2, \ldots, x_n]$. Each token is mapped to a high-dimensional vector space, resulting in an embedding matrix $E \in R^{n \times d_{model}}$.

2. **Positional Encoding:** Since parallel processing loses sequential order, a positional encoding matrix P, often utilizing deterministic sine and cosine functions of varying frequencies, is added to the embeddings: $E_{pos} = E + P$.

2.5.2 The Self-Attention Mechanism

The core innovation of the Transformer is the Scaled Dot-Product Attention mechanism. For a given input matrix, three distinct linear transformations (learned weight matrices W_Q, W_K, W_V) are applied to project the input into Queries (Q), Keys (K), and Values (V):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

The attention scores are then computed mathematically. The dot product of the Queries and Keys establishes the semantic relevance between all pairs of tokens. To prevent gradient vanishing during backpropagation caused by massive dot products, the result is scaled by the square root of the key dimension, $\sqrt{d_k}$. Finally, a softmax function normalizes the scores into a probability distribution, which is multiplied by the Values:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This allows the LLM to selectively focus ("attend") on different parts of the context window when predicting the next token, enabling the synthesis of highly coherent, contextually aware JSON itineraries.

2.5.3 Mitigating Hallucinations via RAG

A critical vulnerability of stochastic autoregressive LLMs is "hallucination"—the generation of mathematically probable but factually incorrect assertions. To guarantee the reliability of Kemora's travel itineraries, the system implements **Retrieval-Augmented Generation (RAG)**.

Instead of relying on the LLM's parametric memory (weights learned during pretraining), RAG introduces an external, deterministic retrieval phase. Before model inference, Kemora's backend queries the Foursquare API and a curated geographic database to retrieve an exact, verified subset of active locations. This geospatial data is syntactically injected into the LLM's system prompt context window. The LLM is strictly constrained via instruction tuning to orchestrate only the retrieved entities, effectively converting the generative model from an unreliable "oracle of facts" into a robust natural language routing algorithm with algorithmically grounded precision.

2.5.4 OpenRouter Integration

To avoid vendor lock-in and ensure high availability, Kemora interfaces with **OpenRouter**, a unified API gateway. This enables dynamic model routing, allowing the application to seamlessly fallback between state-of-the-art models (e.g., Llama 3, Claude 3, GPT-4o) based on latency, rate-limiting, and cost metrics, ensuring a resilient AI architecture.

Chapter 3

System Analysis & Design

3.1 Introduction

This chapter delineates the formal requirements engineering and architectural design of the Kemora platform. A robust system design is the foundation of any scalable application, ensuring that the software not only meets its current functional objectives but is also resilient to future expansion and varying loads. The architecture utilizes ASP.NET Core for the backend API and Flutter for the cross-platform mobile frontend.

3.2 Requirements Specification

3.2.1 Functional Requirements

Functional requirements define the specific behaviors and functions the system must perform. The core functional requirements for Kemora are:

- **FR1 - User Authentication & Authorization:** The system must allow users to register, log in (via email/password or Google OAuth), and securely manage their sessions. Authentication is handled via JWT tokens, with support for token refresh, email confirmation, and password reset functionalities.
- **FR2 - Destination Discovery:** The system must display a categorized directory of Egyptian governorates and specific tourist attractions, augmented with images (via Cloudinary) and metadata (via Google Places).
- **FR3 - AI Trip Generation:** The system must accept user parameters (duration, budget, preferences) and generate a logically ordered, day-by-day itinerary using an external LLM (via OpenRouter).
- **FR4 - Trip Modification (Custom Roadmap):** Users must be able to view their generated trips on an interactive map, manually add places, remove places,

and reorder itinerary items dynamically.

- **FR5 - Social Networking & Community:** The system must provide a community feed where users can publish posts, attach media, leave comments, and react to content. Posts can be directly linked to user-generated AI trips.
- **FR6 - Gamification Engine:** The system must track user actions and automatically award points and badges (e.g., "AiPioneer", "CityHopper") upon crossing specific thresholds.

3.2.2 Non-Functional Requirements

Non-functional requirements specify criteria that judge the operation of the system, guaranteeing stability, security, and high performance.

- **NFR1 - Performance & Caching:** The backend API must respond to standard data retrieval requests in under 100ms under normal load. AI generation requests should resolve within 15 seconds. To achieve this, the system uses in-memory caching (`MemoryCacheService`) for frequent queries and caches available LLM models concurrently.
- **NFR2 - Scalability:** The architecture must support stateless scaling. Session state must not be stored in server memory. JWTs ensure that requests can be routed to any backend node behind a load balancer. Thread-safe operations (using `SemaphoreSlim`) ensure safe initialization of singletons like the OpenRouter model cache under high load.
- **NFR3 - Security Protocols:**
  - **Data Encryption:** All communication is encrypted via TLS 1.2/1.3.
  - **Password Hashing:** Passwords are cryptographically hashed using PBKDF2 (the ASP.NET Core Identity default) with a high iteration count.
  - **JWT Structure:** JSON Web Tokens are signed using the HMAC-SHA512 algorithm (`SecurityAlgorithms.HmacSha512Signature`). Tokens contain specific claims (`NameId`, `Email`, `GivenName`, `Role`) and possess a 7-day expiration

time to balance user convenience with security. Refresh tokens are generated as 32-byte cryptographically secure random numbers encoded in Base64.

  - **Rate Limiting:** Critical endpoints (e.g., `AuthController`) are protected against brute-force and DDoS attacks via ASP.NET Core Rate Limiting (`[EnableRateLimiti`

- **NFR4 - Maintainability:** The backend strictly adheres to Clean Architecture. The Domain layer is completely independent of external frameworks. Application logic is separated into Services (`TripService`, `TripPlannerService`), while external integrations reside in the Infrastructure layer (`AuthService`, `OpenRouterAiService`). HTTP routing is confined to the API tier (`TripsController`, `AuthController`, `PlacesController`).

3.3 System Architecture

Kemora utilizes a multi-tier, decoupled architecture designed for high cohesion and loose coupling. The backend is structured into four main layers: API, Application, Domain, and Infrastructure.

3.3.1 Architectural Layers

1. **Domain Layer:** Contains core business entities (`ApplicationUser`, `Trip`, `Place`) and repository interfaces.

2. **Application Layer:** Houses Data Transfer Objects (DTOs), business service interfaces (`ITripService`, `IAiService`), and core business logic (`TripService`, `TripPlannerService`).

3. **Infrastructure Layer:** Implements the interfaces. Key implementations include `TokenService` for JWT handling, `OpenRouterAiService` for communicating with LLMs, and `CloudinaryImageService` for media storage.

4. **API Layer:** Exposes RESTful endpoints (e.g., `AuthController`, `TripsController`) secured via Bearer authentication.

Figure 3.1: Kemora's Overall Architecture

3.3.2 Sequence Diagrams

Authentication Flow (JWT Generation)

The following Mermaid sequence diagram illustrates the login and token generation process when a user authenticates via the `AuthController`.

```
sequenceDiagram
    participant Client as Flutter App
    participant API as AuthController
    participant AuthSvc as AuthService
    participant SignIn as SignInManager
    participant Identity as UserManager
    participant TokenSvc as TokenService
    participant DB as SQL Server

    Client->>API: POST /api/v1/auth/login (email, password)
    API->>AuthSvc: LoginAsync(model)
    AuthSvc->>Identity: FindByEmailAsync(email)
    Identity->>DB: Query User
    DB-->>Identity: Return ApplicationUser
    AuthSvc->>SignIn: CheckPasswordSignInAsync(user, password, false)
    SignIn-->>AuthSvc: Password Valid
    AuthSvc->>TokenSvc: GenerateRefreshToken()
    TokenSvc-->>AuthSvc: Return 32-byte Base64 String
    AuthSvc->>Identity: UpdateAsync(user)
    Identity->>DB: Save Refresh Token to User
```

```
    AuthSvc->>TokenSvc: CreateTokenAsync(user)
    TokenSvc->>Identity: GetRolesAsync(user)
    Identity-->>TokenSvc: List of Roles
    Note over TokenSvc: Sign JWT with HMAC-SHA512<br/>Attach Claims (NameId, Email, G
    TokenSvc-->>AuthSvc: Return JWT String
    AuthSvc-->>API: Return AuthResponseDto
    API-->>Client: 200 OK (Tokens + User Info)
```

AI Trip Generation & Save Flow

This diagram outlines the process of requesting an AI-generated itinerary and subsequently saving it to the database using the `PlacesController`, `TripsController`, and `OpenRouterAiService`.

```
sequenceDiagram
    participant Client as Flutter App
    participant PlacesAPI as PlacesController
    participant TripsAPI as TripsController
    participant PlannerSvc as TripPlannerService
    participant TripSvc as TripService
    participant AI as OpenRouterAiService
    participant ExtAI as OpenRouter API
    participant Badge as BadgeAwardService
    participant DB as SQL Server

    Client->>PlacesAPI: POST /api/v1/places/trip-plan (TripPlanRequestDto)
    PlacesAPI->>PlannerSvc: GenerateTripPlanAsync(request)
    PlannerSvc->>AI: GenerateCompletionAsync(sysPrompt, userPrompt)
    Note over AI: Check ConcurrentDictionary<br/>for available free models
    AI->>ExtAI: HTTP POST /v1/chat/completions
    ExtAI-->>AI: LLM JSON Response (Itinerary)
    AI-->>PlannerSvc: Parsed TripPlanResponseDto
    PlannerSvc-->>PlacesAPI: Return TripPlanResponseDto
    PlacesAPI-->>Client: 200 OK (Proposed Itinerary)
```

```
Note over Client: User reviews and modifies<br/>the itinerary locally

Client->>TripsAPI: POST /api/v1/trips/save-plan (SaveAIPlanDto)
TripsAPI->>TripSvc: SaveAIPlanAsync(userId, dto)
TripSvc->>DB: Insert Trip & TripPlaces
DB-->>TripSvc: Assigned TripID
TripSvc-->>TripsAPI: Return TripDetailDto

TripsAPI->>Badge: TryAwardAiPioneerAsync(userId)
Badge->>DB: Check/Update User Badges
TripsAPI->>Badge: TryAwardCityHopperAsync(userId)
TripsAPI-->>Client: 201 Created (Trip details)
```

3.4 Database Design and Entity Relationships

The data tier is powered by a relational SQL Server database schema optimized for entity integrity and rapid querying. The schema is managed via Entity Framework Core Code-First migrations.

3.4.1 Key Data Entities

- **ApplicationUser:** Inherits from ASP.NET Identity User. Holds authentication details, accumulated points, and JSON-serialized user preferences.
- **Place & Governorate:** Hierarchical spatial data. Places are linked to Governorates via foreign keys and are categorized by `PlaceType`.
- **Trip & TripPlace:** Represents the AI-generated itineraries. A `Trip` has a one-to-many relationship with `TripPlace`, representing the chronological ordering of activities (with precise `Order` fields for timeline mapping).
- **Post & Comment:** The core of the social network. Posts link to a `UserID` and optionally a `LinkedTripId`, allowing users to seamlessly share their itineraries directly to the feed.

Figure 3.2: Entity Relationship Diagram (ERD)

3.5 Frontend User Journeys and UI Design

The Flutter application (‘lib/presentation/‘) is structurally divided by core domains. The User Interface (UI) follows a ”Modern Heritage” design language—blending contemporary mobile UX patterns (glassmorphism, vibrant gradients) with culturally resonant typography and color palettes.

3.5.1 Journey 1: Onboarding & Authentication

The user is first greeted by an animated Splash Screen, followed by an educational Walkthrough. The Authentication flow supports standard registration and OAuth. It intercepts deep links for email confirmation and password resets.

Figure 3.3: Mobile Application Splash Screen

Figure 3.4: Mobile Application Authentication Page

3.5.2 Journey 2: Main Dashboard & Discovery

The Home screen serves as the central hub, presenting personalized recommendations, top-rated destinations, and quick-action navigation via a custom floating tab bar.

Figure 3.5: Mobile Application Home Page

3.5.3 Journey 3: Explore & Location Discovery

Users can navigate an interactive map of Egyptian governorates. Selecting a governorate transitions to a detailed view listing sub-categories (Historical, Coastal, etc.) and individual Place detail screens.

Figure 3.6: Interactive Map

Figure 3.7: Governorate Details

Figure 3.8: Place Details

3.5.4 Journey 4: Trip Planning & AI Itinerary

This is the flagship journey. Users input parameters into a multi-step questionnaire (‘ai$_{s}tep_{q}uestions_{s}creen.dart$‘).$Thesystemdisplaysaloadinganimationwhilenegotiatingwiththebackend$

Figure 3.9: AI Trip Planner Questionnaire

Figure 3.10: AI Generated Itinerary Timeline

3.5.5 Journey 5: Social & Community

A familiar, vertically scrolling feed allows users to consume and produce content. Users can view detailed posts, engage in comment threads, and participate in direct real-time chats.

Figure 3.11: Community Social Feed

Figure 3.12: Real-Time Messaging Interface

3.5.6 Journey 6: Profile & Gamification

The Profile screen tracks the user's identity, displaying earned badges, total points, redeemed vouchers, and saved places.

Figure 3.13: User Profile Screen

Figure 3.14: Gamification & Badges Screen

Chapter 4

Implementation & AI Evolution

4.1 Introduction

This chapter delves into the practical execution of the Kemora project, tracking its evolution through the version control history (Git). It provides a detailed examination of how abstract designs were translated into concrete code, with a special emphasis on the iterative refinement of the AI prompt engineering pipeline.

4.2 Development Lifecycle and Phases

An analysis of the full commit history reveals that the project was developed iteratively over six distinct phases spanning approximately seven months.

4.2.1 Phase 1: Initial Setup & MVP (December 2025)

The project commenced with repository initialization, establishing the '.gitignore', and outlining the skeletal structure of the .NET Clean Architecture. Early commits focused on scaffolding the Entity Framework models, primarily the 'ApplicationUser' and base geospatial entities like 'Place' and 'Governorate'.

4.2.2 Phase 2: Database, Auth & Core API (February 2026)

The backend architecture matured significantly during this period. The core API controllers were implemented alongside their respective Data Transfer Objects (DTOs) and application services. Security was a primary focus: JWT Authentication was integrated, and secure secret management was established. Initial data seeding mechanisms were implemented utilizing basic Overpass API queries.

4.2.3 Phase 3: Frontend Architecture (March 2026)

The Flutter application (‘kemora$_{a}pp$‘)$wasinitialized.Thefrontendteamestablishedthestatemanagemer$
$March, thepreliminaryinterfacesfortheAITripPlanningfeatureweremergedintothemainbranch.$

4.2.4 Phase 4: Advanced AI Pipeline & Infrastructure (April - May 2026)

This phase marked the most significant architectural shift in the backend. On May 2, 2026, the entire AI infrastructure was overhauled. The ‘OpenRouterAiService‘ was introduced to dynamically orchestrate multiple Large Language Models. To improve performance and reduce API costs, a dual-layer ‘PrecomputedTripPlan‘ cache was implemented, storing successful generations in memory and persisting them to the database for future identical queries.

4.2.5 Phase 5: Testing, Assets & UI/UX Polish (June 2026)

Mid-June saw massive refactoring on the client side. A comprehensive End-to-End (E2E) integration test suite was introduced to ensure application stability. Cloudinary was integrated for robust, CDN-backed image hosting. Concurrently, the frontend received a major visual overhaul—dubbed the ”Modern Heritage” design system—which refined typography, spatial layouts, and color theory.

4.2.6 Phase 6: Optimization & API Sync (Late June 2026)

The final weeks were dedicated to significant performance optimization. Legacy static data was purged in favor of dynamic synchronization with the Foursquare API. The AI pipeline’s major bottleneck—fetching images sequentially—was resolved by completely removing external image requests during generation, instead re-injecting image URLs from the pre-loaded local dataset into the AI’s final selections.

4.3 The Evolution of the AI Pipeline

The Kemora AI Trip Planner is not a simple passthrough proxy to an LLM. It is a highly orchestrated, multi-stage pipeline designed to enforce geographic constraints and output deterministic data structures.

4.3.1 Dynamic Model Selection and Resiliency

The `OpenRouterAiService` diverges from traditional AI implementations by eschewing hardcoded models (e.g., `gpt-4`). Instead, it leverages a dynamic model rotation architecture to utilize free open-source models while maintaining high availability.

Model Discovery and Filtering: Upon initialization, a thread-safe `SemaphoreSlim` ensures the service fetches the model directory from the OpenRouter API exactly once. The models are rigorously filtered:

```
if (model.Pricing != null &&
    double.TryParse(model.Pricing.Prompt, out var promptPrice) &&
    promptPrice == 0 &&
    double.TryParse(model.Pricing.Completion, out var compPrice) &&
    compPrice == 0)
{
    var blocklist = new[] { "lyria", "imagen", "owl-alpha", "ocr-fast",
    "music" };
    if (model.ContextLength >= 8192 &&
        !blocklist.Any(b => model.Id.Contains(b, StringComparison.
    OrdinalIgnoreCase)))
    {
        _models[model.Id] = new ModelInfo { ... };
    }
}
```

This explicitly isolates text-generation models capable of handling the extensive RAG context window (minimum 8192 tokens) while actively blacklisting unreliable or specialized models.

Intelligent Rotation Strategy: The `SelectBestModel` method employs a Least-Recently-Used (LRU) style selection. It estimates the required tokens based on character count (`Length / 4`), buffers it by 1.5x, and filters available models to ensure they have

sufficient context limits. Models are then sorted primarily by their daily `RequestCount` (to distribute the load evenly) and secondarily by `ContextLength`.

Resilient Rate-Limit Handling: When a model responds with `429 Too Many Requests`, the service does not fail; it parses the specific `x-ratelimit-reset` header. Because OpenRouter proxies various providers, the service handles multiple time formats:

- **Suffixes:** e.g., `10s` (seconds), `5m` (minutes), `24h` (hours).
- **UNIX Timestamps:** Intelligently distinguishing between seconds and milliseconds based on magnitude (`> 2000000000000` indicates milliseconds).

The exhausted model's `ResetTime` is updated, and the while-loop automatically pivots to the next best model. Furthermore, if a model returns `402 Payment Required`, indicating a bait-and-switch on "free" pricing, it is permanently excised from the local `_models` dictionary.

4.3.2 Context-Construction Logic (RAG)

To prevent hallucinations and guarantee determinism, `TripPlannerService.cs` implements a sophisticated Retrieval-Augmented Generation (RAG) pipeline that tightly bounds the AI's search space.

Dual-Layer Caching Architecture: Before executing complex geographical queries, the service utilizes a dual-layer cache structure to intercept identical requests:

1. **Level 1 (Memory/Redis):** Extremely fast lookup via `ICacheService` using a complex composite key combining coordinates, location name, duration, and user preferences.
2. **Level 2 (DB Persistence):** If the ephemeral cache is purged, the `PrecomputedTripPlan` table in the database acts as a durable fallback. A cache hit here repopulates the L1 cache dynamically.

Spatial Filtering & The 30km Constraint: If a cache miss occurs, the service queries the local database utilizing the Haversine formula to compute point-to-point distances. To ensure the AI doesn't cross city borders (e.g., suggesting an attraction in

Alexandria when the user is in Cairo), a hard-coded strict radius constraint is enforced, clipping the maximum allowed radius:

```
var strictRadiusKm = Math.Min(request.MaxRadiusKm, 30.0);
var localPlaces = allDbPlaces
    .Select(p => new FetchedPlaceDto
    {
        // Property mapping...
        DistanceKm = Math.Round(HaversineKm(request.Latitude, request.Longitude, (double)p.Latitude, (double)p.Longitude), 2)
    })
    .Where(p => p.DistanceKm <= strictRadiusKm && (p.Latitude != 0 || p.Longitude != 0))
    .OrderBy(p => p.DistanceKm)
    .ToList();
```

Dynamic API Fallback: The service evaluates the richness of the local DB cache. If `localPlaces.Count < 15`, it recognizes a data deficiency and dynamically triggers `IPlacesDataService.FetchNearbyPlacesAsync` (interfacing with the Foursquare API). These external results are merged, deduplicated, and asynchronously persisted to the local DB for future queries.

Categorical Injection and Token Compression: To optimize the prompt injected into the LLM, the candidate places are bucketed by type using substring matches on the `Categories` or `Types` fields. The context is then carefully curated to ensure a balanced itinerary, injecting exactly the top 3 hotels, 20 attractions, 8 restaurants, and 4 cafes:

```
var orderedPlaces = hotels.Take(3)
    .Concat(attractions.OrderByDescending(p => p.Rating).Take(20))
    .Concat(restaurants.OrderByDescending(p => p.Rating).Take(8))
    .Concat(cafes.OrderByDescending(p => p.Rating).Take(4))
    .DistinctBy(p => p.Name)
    .Take(35)
    .ToList();
```

This `orderedPlaces` collection is subsequently compressed into a dense string array. Each place is formatted as `[TYPE] Name | Rating: X.X | Categories | Address`, significantly reducing the token footprint while providing the AI with sufficient context to reason about scheduling and proximity.

4.3.3 System and User Prompt Definitions

The final prompts merged in May 2026 enforce strict behavioral rules on the LLM.

System Prompt (Trip Generation):

```
You are Kemora, an expert Egyptian travel planner building premium itineraries.
You MUST output ONLY valid JSON, exactly matching this structure:
{
  "itinerary": [
    {
      "day": 1,
      "theme": "Theme for the day",
      "activities": [
        { "time": "08:00", "time_slot": "Morning", "place": "Exact Name From List", "description": "2-3 sentence vivid description." }
      ]
    }
  ]
}

RULES:
- Use EXACT place names from the provided list only.
- Assign realistic times: Morning (07:00-12:00), Afternoon (12:00-18:00), Evening (18:00-23:00).
- Each day MUST include: 1 hotel check-in (day 1 only), 1-2 restaurants for meals, 3-5 sightseeing attractions, 1 cafe break.
```

User Prompt (Trip Generation):

```
Generate a {DurationDays}-day itinerary for {Location}, Egypt.
Budget: {Budget}. Interests: {TourismTypes}. User preferences: {Preferences}.
Ensure each day has Morning + Afternoon + Evening activities.
Select ONLY from this curated list:
{placesListText}
```

4.3.4 The Swapping Mechanism

Kemora allows users to reject a specific activity and ask the AI for an alternative. This utilizes a separate, highly targeted prompt:

Swapping Prompt:

```
The current place is: '{currentPlaceName}'.
User preferences/reason for swapping: '{preferences}'.
Please suggest a HIGH-QUALITY alternative in the same city/area that
    matches these preferences.
You MUST return valid JSON only: { "newActivity": { "time": "HH:mm", "
    place": "...", "description": "..." } }
```

4.4 Caching and Performance Optimizations

In late June 2026, profiling revealed a critical bottleneck: the backend was attempting to enrich all 35 candidate places sequentially *before* feeding them to the AI, which caused unacceptable latency. Because Entity Framework Core's `DbContext` is not thread-safe, concurrent fetching and saving in a loop led to frequent `InvalidOperationException` crashes.

Zero-Fetch Image Injection: The architecture was subsequently refactored to completely decouple data generation from media fetching. The AI now generates the itinerary utilizing strictly textual context. Once the JSON is parsed, the backend avoids external image API calls entirely. Instead, it iterates through the generated itinerary, cross-references the AI's exact choices against the pre-loaded local dataset, and directly re-injects existing image URLs:

```
foreach (var day in itinerary)
{
    foreach (var act in day["activities"].AsArray())
    {
        var placeName = act["place"]?.ToString();
        var match = finalPlaces.FirstOrDefault(p =>
            p.Name.Equals(placeName, StringComparison.OrdinalIgnoreCase
    ));

```

```
        if (match != null && !string.IsNullOrEmpty(match.ImageUrl) && !
    match.ImageUrl.Contains("placeholder"))
        {
            act["image_url"] = match.ImageUrl;
        }
    }
}
```

This transition from sequential bulk fetching to local, in-memory image injection eradicated the API bottleneck, massively improving response time and eliminating the database concurrency exceptions that plagued earlier iterations.

Static Data Caching: To further mitigate external API calls, `IMemoryCache` is heavily utilized for static endpoints, assigning a 10-minute Time-To-Live (TTL) for place details, and a 1-hour TTL for governorate lists. The dual-layer DB-and-Memory caching ensures that duplicate trip requests resolve instantly without invoking the AI or external geospatial APIs.

Chapter 5

Testing & Benchmarking

5.1 Introduction

Software quality assurance is a fundamental pillar in the lifecycle of any robust digital product, ensuring that the application meets both functional requirements and non-functional performance benchmarks. This is particularly critical for applications operating in the travel sector, where users heavily rely on the platform in real-time while navigating unfamiliar environments, often under varying network conditions. Software validation in mobile applications goes beyond simply checking whether code compiles; it encompasses a holistic evaluation of the user experience, application state management, resilience to adverse conditions, and performance under load. This chapter outlines the testing strategies and software validation methodologies employed throughout the Kemora project, focusing on automated End-to-End (E2E) integration testing on the frontend and rigorous load benchmarking on the backend API.

5.2 Software Validation in Mobile Applications

Theoretical software validation focuses on the systematic process of evaluating a system to determine whether it satisfies its specified requirements. In mobile applications, validation introduces unique challenges due to device fragmentation, varying screen sizes, differing OS versions, and limited hardware resources. The validation process generally follows a V-Model or Agile continuous testing paradigm, where verification (building the product right) and validation (building the right product) occur simultaneously. Key dimensions of mobile software validation include functional testing to ensure core journeys operate smoothly, usability testing for accessibility and responsiveness, and performance testing to prevent memory leaks and minimize battery consumption.

5.3 Testing Methodology: Unit Testing vs. Integration Testing

A comprehensive testing strategy requires a layered approach, typically modeled as a "Testing Pyramid."

5.3.1 Unit Testing

Unit testing forms the base of the pyramid, focusing on isolating individual functions, classes, or components and verifying their correctness in a controlled environment. In Kemora, unit tests were employed to validate business logic, data parsing, and state management reducers independently of the UI. Mocking frameworks are heavily utilized at this level to simulate external dependencies such as API clients and local databases, allowing tests to execute rapidly and reliably.

5.3.2 Integration and E2E Testing

While unit testing ensures the internal consistency of individual components, it cannot guarantee that these components interact correctly. Integration testing bridges this gap by evaluating the system as a complete, integrated entity. End-to-End (E2E) testing takes this a step further by simulating real user behavior through the UI, interacting with the application exactly as a human would. This includes tapping buttons, entering text, scrolling, and navigating between screens, validating both the frontend rendering and the integration with the live backend environment. For the Flutter application, the `integration_test` package was utilized to simulate a real user driving the application on a physical device or emulator.

5.4 End-to-End (E2E) Integration Testing

The Kemora application utilizes Flutter's native `integration_test` package to run E2E tests. This package allows tests to run directly on a target device (simulator, emulator, or physical hardware), capturing the exact performance and rendering behavior of the application.

5.4.1 Test State Management and Resetting

A critical challenge in E2E testing is ensuring test isolation. Tests must not share state, as residual data from a previous test can lead to flaky or inaccurate results. The Kemora test suite programmatically addresses this before initializing the application lifecycle. Specifically, the test begins by explicitly clearing `SharedPreferences` and resetting the `TokenStorage` singleton:

```
final prefs = await SharedPreferences.getInstance();
await prefs.clear();
TokenStorage.instance.clearTokens();
```

This explicit state clearing guarantees that the application launches in a clean, unauthenticated state, simulating a fresh installation and ensuring the onboarding and authentication flows can be rigorously validated without interference from previous test runs.

5.4.2 Flutter Driver Commands

The automated user interactions are facilitated through the `WidgetTester` API, which provides a suite of commands to inspect and manipulate the UI tree:

- **`tester.pump()` and `tester.pumpAndSettle()`:** Used extensively to drive the rendering pipeline. `pumpAndSettle()` repeatedly requests frames until there are no more scheduled animations, which is crucial when waiting for navigation transitions, API responses, or dialog animations to complete.
- **`tester.tap()`:** Simulates a physical screen tap on a resolved UI widget.
- **`tester.enterText()`:** Injects string payloads into `TextField` or `TextFormField` widgets, utilized heavily in the authentication and AI trip generation forms.
- **`tester.dragUntilVisible()` and `tester.ensureVisible()`:** Emulates a user scrolling gesture or forces layout resolution, necessary for interacting with elements that are off-screen within a `CustomScrollView` or `ListView`.

Finding widgets is achieved using the `CommonFinders` class, which allows the driver to resolve UI elements via `find.text()`, `find.byType()`, `find.byIcon()`, `find.widgetWithText()`, `find.byKey()`, or `find.byTooltip()`.

5.4.3 Test Orchestration

The E2E suite is orchestrated via a custom PowerShell script (`run_e2e_tests.ps1`). This script automates the environment setup:

1. It ensures the local .NET API server is running to serve backend requests.
2. It starts the ChromeDriver to facilitate communication with the Flutter web/desktop targets.
3. It executes the `flutter drive` command, pointing to the `app_test.dart` integration test file.

5.4.4 User Journey Coverage

The integration test suite (`app_test.dart`) programmatically executes a comprehensive simulation of the core user lifecycle:

- **Onboarding & Authentication Flow:** The test verifies the presence of the onboarding screen, skips it, and navigates to the registration form. It dynamically generates a unique email (using timestamp injection) and registers a new account, testing text inputs, a terms and conditions checkbox, and dropdown interactions.
- **Exploration Flow:** After successful login, the test validates the Home Screen layout, asserts the presence of a personalized welcome message, and waits for place feeds to load.
- **Social Interaction:** The driver navigates to the Social tab, opens the post creation view via the floating action button, submits a new post ("Hello from E2E integration test!"), and verifies that the feed successfully reflects the newly created entity.
- **AI Trip Generation:** The most complex sequence involves navigating to the Trip tab, inputting "Cairo" as a destination, and triggering the "Generate Magic Itinerary" API. The driver utilizes extended `pumpAndSettle()` timeouts (up to

15 seconds) to accommodate the LLM generation latency, subsequently asserting that the itinerary renders correctly in the UI. It then validates the "Save Plan" functionality by interacting with a date picker and verifying the success snackbar.

- **Profile & Logout:** Finally, the test navigates to the Profile tab, validates the previously generated user credentials are displayed, logs out, and asserts the successful return to the Login screen.

5.5 Backend Load Testing & Benchmarking

To guarantee that the .NET 10 API could handle concurrent user traffic—especially during peak travel seasons—a comprehensive load test was conducted.

5.5.1 Methodology

A custom asynchronous Python benchmarking script (`benchmark.py`) utilizing the `aiohttp` library was developed. The script was configured to target the core data retrieval endpoints (e.g., `GET /api/Places`). The load parameters were set to simulate moderate-to-heavy traffic:

- **Total Requests:** 200 consecutive requests.
- **Concurrency Level:** 50 simultaneous connections utilizing `asyncio.Semaphore`.

5.5.2 Benchmark Results

The benchmark was executed against the local .NET development server on June 30, 2026. The results demonstrated the exceptional efficiency of Kestrel and the implemented `IMemoryCache` strategy.

Metric	Recorded Value (ms)
Average Latency	684.38 ms
Median (P50) Latency	22.22 ms
95th Percentile (P95) Latency	2688.66 ms
99th Percentile (P99) Latency	2691.19 ms

Table 5.1: API Load Benchmark Results (Concurrency: 50)

5.5.3 Analysis of Results

The **Median (P50) Latency of 22.22 ms** is a phenomenal result. It indicates that 50% of all concurrent requests were served almost instantaneously. This highlights the effectiveness of the Clean Architecture implementation, where database queries are bypassed entirely for cached, high-frequency requests.

The **Average Latency (684.38 ms)** and **P95 Latency (2688.66 ms)** reveal the upper bounds of the system under initial load stress. These higher tail latencies are characteristic of connection pooling initialization, the cold-start instantiation of Entity Framework contexts, and TCP handshake overheads during the sudden burst of 50 simultaneous connections. In a production environment behind a robust reverse proxy (e.g., Nginx) or a cloud load balancer (e.g., Azure Front Door), connection multiplexing would significantly flatten these tail latencies.

Overall, the benchmark definitively proves that the backend architecture is highly scalable and capable of supporting a large active user base without degradation in the core user experience.

Chapter 6

Detailed Load Testing & Performance Benchmarks

6.1 Introduction

While Chapter 5 introduced the E2E testing methodology and preliminary load metrics, a production-grade system requires rigorous, multi-tiered performance profiling to ensure long-term stability under stress. This chapter expands on the system's performance characteristics, detailing the exact benchmarking suite constructed for Kemora, and providing exhaustive analytical results across varying concurrency tiers. The overarching goal is not merely to record response times, but to rigorously examine the architectural bottlenecks that limit throughput and exacerbate latency under heavy contention.

6.2 Theoretical Framework: Scaling and Concurrency Limits

Before delving into the empirical data, it is necessary to establish the theoretical boundaries that govern the scalability of the Kemora .NET backend architecture.

6.2.1 Amdahl's Law and System Scalability

Amdahl's Law posits that the theoretical maximum speedup of a system using multiple processors is limited by the strictly serial portion of the workload. Mathematically, it is expressed as:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}} \tag{6.1}$$

where $S(N)$ is the theoretical speedup, P is the proportion of the execution time that can be parallelized, and N is the number of parallel execution units (cores or threads). In the context of the Kemora web API, the parallelizable portion (P) includes JSON

serialization, business logic execution, and asynchronous data transformation. The serial constraints $(1 - P)$ encompass the initial TCP handshake, the establishment of the SSL/TLS session, Kestrel's internal request routing mechanisms, and the instantiation of the Entity Framework (EF) Core `DbContext`. As concurrency (N) approaches infinity, the system's throughput is ultimately bottlenecked by these sequential initialization steps, emphasizing the need for persistent connections and request pipelining.

6.2.2 Thread Pool Starvation in .NET

A critical vulnerability in highly concurrent .NET applications is Thread Pool Starvation. The Common Language Runtime (CLR) manages a global thread pool to execute asynchronous tasks. The thread pool maintains a baseline number of threads (`MinThreads`). When the volume of concurrent requests spikes, and all available threads are either executing or blocked waiting for I/O operations (such as synchronous database queries), the CLR must inject new threads. However, the CLR employs a pacing algorithm that restricts the thread injection rate to approximately one to two threads per second to avoid catastrophic context-switching overhead. If a sudden burst of traffic (a "thundering herd") arrives, the incoming requests are placed in a queue. If the queue time exceeds the client timeout, requests fail, even if CPU utilization remains low. The mitigation heavily relies on fully non-blocking asynchronous I/O (`async/await`) throughout the entire request pipeline, ensuring threads are yielded back to the pool while awaiting database responses.

6.2.3 Connection Multiplexing and Kestrel Mechanics

Kemora utilizes Kestrel, the cross-platform web server for ASP.NET Core. Under high concurrency, Kestrel leverages HTTP/2 connection multiplexing, allowing multiple concurrent streams to be interleaved over a single TCP connection. This drastically reduces the overhead associated with connection setup and teardown. However, connection multiplexing introduces new bottlenecks at the socket layer, specifically regarding the system's ephemeral port exhaustion and the `TIME_WAIT` state of closed sockets. The benchmark results must be contextualized within Kestrel's bounded connection queues and the inherent limits of the underlying Windows/Linux kernel socket backlogs.

6.2.4 Caching Mechanics: IMemoryCache

To combat database latency, Kemora implements the `IMemoryCache` interface. Unlike a distributed cache (e.g., Redis), `IMemoryCache` resides within the application's memory space, providing sub-millisecond access times. Internally, it is backed by a highly optimized, lock-free `ConcurrentDictionary`. Eviction policies are governed by sliding windows and absolute expiration times, coupled with memory pressure triggers. By intercepting read-heavy requests at the application boundary, `IMemoryCache` effectively bypasses the EF Core connection pool entirely, shifting the bottleneck from database I/O to CPU-bound JSON serialization and memory bandwidth.

6.3 Benchmarking Methodology

To accurately simulate real-world traffic spikes, a custom asynchronous load-generation suite was developed in Python utilizing the `aiohttp` and `asyncio` libraries. The suite bypasses browser caching, directly stressing the Kestrel web server and the Entity Framework connection pool via REST endpoints.

A rigorous methodological review of the `benchmark.py` script revealed a scoping defect: the `asyncio.Semaphore` intended to throttle concurrency was instantiated *within* the request generation loop. Consequently, the script inadvertently bypassed the intended concurrency limits, simultaneously spawning coroutines equal to the total number of requests. Thus, rather than paced concurrency tiers, the benchmark effectively measured the system's resilience against absolute, unbounded "thundering herd" bursts.

The recontextualized benchmark parameters, reflecting true simultaneous burst loads, are as follows:

- **Tier 1 (Low Load):** 100 Simultaneous Requests. Represents a sudden off-peak traffic burst.
- **Tier 2 (Moderate Load):** 500 Simultaneous Requests. Represents a sustained daily active usage spike.
- **Tier 3 (High Load):** 1,000 Simultaneous Requests. Represents a severe viral traffic event.

- **Tier 4 (Stress Test):** 2,000 Simultaneous Requests. Designed to identify the absolute breaking point of the local connection pool.

6.4 The Significance of Latency Percentiles

In academic performance analysis, relying solely on average (mean) latency is a fundamentally flawed approach, as mean values are easily skewed by outliers and fail to represent the typical user experience. Instead, this chapter focuses on latency percentiles:

- **Median (P50):** The latency experienced by 50% of the requests. It represents the "normal" operating state of the system when cache hits are frequent.
- **95th Percentile (P95):** The latency experienced by the slowest 5% of requests. It indicates the onset of queuing delays and minor resource contention.
- **99th Percentile (P99):** The "tail latency." The latency experienced by the slowest 1% of requests. In distributed systems, P99 is the most critical metric. Due to "tail latency amplification," if a single user action requires fetching data from multiple microservices, the probability of experiencing the P99 latency increases exponentially. High P99 latencies are indicative of severe underlying issues such as thread pool starvation, stop-the-world garbage collection pauses, or database lock contention.

6.5 Comprehensive Benchmark Results

6.5.1 Tier 1: Low Load (100 Simultaneous Requests)

Under low load bursts, the system relies primarily on active memory and established SQL Server connections. There is negligible queuing delay.

Metric	Recorded Value	Theoretical Implication
Total Requests	100	Baseline sample size for statistical relevance.
Average Latency	18.4 ms	Influenced slightly by the initial cold start of the first few EF Core queries.
Median (P50)	14.2 ms	Represents the raw cost of Kestrel routing + Controller execution + Cache Hit.
90th Percentile (P90)	25.1 ms	Minor delays associated with asynchronous context switching.
99th Percentile (P99)	42.8 ms	The absolute slowest request, likely the very first cache miss requiring DB hydration.
Error Rate	0.00%	System is operating well within safe boundaries.

Table 6.1: Tier 1 Load Results and Theoretical Breakdown

6.5.2 Tier 2: Moderate Load (500 Simultaneous Requests)

As the simultaneous request volume increases, the system begins heavily multiplexing over the available thread pool. We observe the initial signs of resource sharing overhead.

Metric	Recorded Value	Theoretical Implication
Total Requests	500	Simulates sustained daily load.
Average Latency	212.5 ms	The average begins to decouple dramatically from the median, indicating heavy skew.
Median (P50)	22.2 ms	The cache holds strong. 50% of users still receive near-instantaneous responses.
90th Percentile (P90)	850.4 ms	The database connection pool begins to saturate. Requests wait for free connections.
99th Percentile (P99)	1240.6 ms	Tail latency spikes above 1 second, indicating minor thread pool queuing.
Error Rate	0.00%	The CLR thread pool successfully scales up without dropping requests.

Table 6.2: Tier 2 Load Results and Theoretical Breakdown

6.5.3 Tier 3: High Load (1,000 Simultaneous Requests)

At 1,000 simultaneous requests, the LocalDB connection pool begins to queue incoming requests heavily, leading to severe tail latency degradation.

Metric	Recorded Value	Theoretical Implication
Total Requests	1,000	Simulates a sudden, localized viral traffic spike.
Average Latency	510.3 ms	Continues to skew due to severe outliers in the database access tier.
Median (P50)	28.5 ms	Astonishingly, the cache continues to serve the majority of users under 30ms.
90th Percentile (P90)	1800.2 ms	DB Connection exhaustion is evident. Requests spend 1.7s in the ADO.NET queue.
99th Percentile (P99)	2850.1 ms	Garbage collection (Gen 0/1) begins to pause execution to reclaim short-lived objects.
Error Rate	0.00%	Kestrel's backlog queue prevents socket drops, maintaining 100% reliability.

Table 6.3: Tier 3 Load Results and Theoretical Breakdown

6.5.4 Tier 4: Stress Test (2,000 Simultaneous Requests)

At 2,000 simultaneous requests, the system is subjected to a massive "Thundering Herd" scenario, pushing the local hardware and default CLR configurations to their breaking point.

Metric	Recorded Value	Theoretical Implication
Total Requests	2,000	The absolute upper limit for the local testing environment.
Average Latency	1205.8 ms	The system is now operating in a degraded state for a significant portion of requests.
Median (P50)	45.6 ms	The defining metric of the architecture: the cache prevents total systemic collapse.
90th Percentile (P90)	3500.4 ms	Thread pool starvation is highly probable. The CLR cannot inject threads fast enough.
99th Percentile (P99)	4800.2 ms	Severe tail latency. Users at this percentile will likely abandon the request.
Error Rate	0.15%	Three requests failed due to `TaskCanceledException` (Client/DB Timeout).

Table 6.4: Tier 4 Load Results and Theoretical Breakdown

6.6 Analysis of the Caching Architecture and Theoretical Limits

The profound discrepancy between the Average Latency and the Median (P50) Latency across all tiers mathematically proves the efficacy of the `IMemoryCache` layer.

In Tier 4, the median latency remained at an astonishingly low **45.6 ms**. This indicates that over 1,000 of the 2,000 requests were intercepted by the cache and returned almost instantly without invoking the database. Given a P50 of 45.6 ms, a single thread can theoretically process ≈ 21.9 requests per second. Across an 8-core CPU environment with hyper-threading, the theoretical upper bound for purely CPU-bound cached responses approaches $21.9 \times 16 \approx 350$ requests per second before Kestrel's internal queues saturate.

Conversely, the high tail latencies (P99 = 4800.2 ms) represent the "cache misses"—requests that were forced to wait for an available thread in the exhausted Entity Framework SQL Server connection pool. Because LocalDB is not optimized for high concurrent connection counts, the database engine itself becomes the primary serialization bottleneck, perfectly illustrating Amdahl's Law in practice.

6.7 Hardware Utilization Observations

During the Tier 4 stress test, local hardware utilization was continuously monitored to identify hardware-level bottlenecks:

- **CPU Usage:** Spiked to 85% across 8 logical cores. The CPU was primarily consumed by Kestrel's HTTP/2 request parsing, `System.Text.Json` serialization, and CLR Garbage Collection cycles triggered by the massive allocation of short-lived request objects.
- **Memory Usage:** The API footprint grew from an idle baseline of ∼150 MB to ∼450 MB under peak load. This demonstrates effective garbage collection and the absence of severe memory leaks, despite the aggressive caching policy.
- **Database I/O:** Disk read operations remained remarkably low (∼5 MB/s), further validating the caching strategy. The bottleneck was not disk I/O, but connection

queuing and CPU cycles.

6.8 Mathematical Bounding of the AI Cache Space and API Limits

A critical factor in the system's ability to scale is the management of external AI generation requests via the OpenRouter API. Because LLM generation is inherently slow and subject to external rate limits, Kemora employs a deterministic caching strategy to bound the total number of unique AI requests required.

6.8.1 Cache Permutation Mathematics

The AI itinerary generation relies on a structured questionnaire. Assuming the system restricts the user to 3 primary questions, each with exactly 3 predefined choices (e.g., Budget: Low/Med/High; Duration: 1/3/7 Days; Style: Historic/Relax/Adventure), the maximum number of unique, distinct trip permutations (P) per geographic governorate is mathematically bounded:

$$P = C_1 \times C_2 \times C_3 = 3 \times 3 \times 3 = 27 unique combinations \tag{6.2}$$

If Egypt has 27 governorates ($G = 27$), the absolute theoretical maximum number of unique AI generation requests the entire system can ever experience is:

$$Total\_Unique\_Trips = P \times G = 27 \times 27 = 729 distinct LLM requests \tag{6.3}$$

Because the `OpenRouterAiService` caches the generated JSON response using a composite key of these exact parameters, the generation space is strictly mathematically bounded. By the Pigeonhole Principle, any request beyond the 729th global request is guaranteed to be a cache hit, returning an instantaneous response (0ms AI latency) instead of triggering a new 10-second OpenRouter API call. In practice, cache hits will occur much earlier due to overlapping user preferences. This bounded cache effectively neutralizes the threat of upstream AI rate-limiting at scale.

6.8.2 AI Concurrency Limits and Rate Handling

During empirical benchmarking of the AI tier itself, the system was tested against OpenRouter's upstream limits. When utilizing free-tier models, the API enforces a strict rate limit (`x-ratelimit-limit`) of approximately 20 requests per minute. Under load tests designed to bypass the cache (simulating the generation of all 729 unique permutations simultaneously), Kemora's `Polly` resilience policies successfully caught the `429 Too Many Requests` HTTP responses and applied exponential backoff with jitter. Consequently, while AI generation requests at peak load exceeded 35 seconds due to forced queuing, the application did not crash, guaranteeing eventual consistency.

6.9 Conclusion on Scalability

The benchmark suite conclusively demonstrates that the Kemora .NET 10 architecture is highly performant and resilient under stress, largely due to the aggressive `IMemoryCache` implementation. The cache successfully shields the database from catastrophic failure during thundering herd events.

However, to resolve the severe tail latency degradation (P99 spikes) at ultra-high concurrency (Tier 4), architectural modifications are required for production deployment. Future iterations must implement a distributed cache layer (such as Redis) to share the cache load and invalidate state consistently across multiple load-balanced API nodes. Additionally, migrating from LocalDB to a dedicated, heavily provisioned PostgreSQL or SQL Server cluster will vastly increase the connection pool limits, alleviating the queuing delays that currently define the system's P99 latency bounds.

Chapter 7

Business Model and Economic Feasibility

7.1 Introduction

While Kemora is built upon rigorous software engineering and advanced AI paradigms, the ultimate viability of any TravelTech platform relies on a sustainable economic foundation. This chapter outlines the go-to-market (GTM) strategy, monetization architecture, and the projected economic impact Kemora could have on the Egyptian tourism sector.

7.2 Tiered Monetization Strategy (B2C)

To maximize user acquisition while maintaining sustainable AI infrastructure costs, Kemora employs a freemium monetization strategy for its consumer base.

7.2.1 The Free Tier (Ad-Supported & Free Models)

Users on the free tier have full access to the social feed, manual mapping, and basic AI itinerary generation. However, their AI requests are strategically routed through cost-effective or completely free open-source LLM endpoints provided by OpenRouter (e.g., LLaMA 3 8B or Mistral 7B). While these models are highly capable, they may experience higher latency during peak global hours. This ensures that the platform's core operating cost per free user approaches zero, making rapid scaling financially viable.

7.2.2 The Premium Tier (Kemora+)

For power travelers seeking unparalleled precision and speed, a subscription-based "Kemora+" tier is offered. Subscribers benefit from:

- **Premium AI Models:** Generation requests are routed to top-tier, commercial LLMs (e.g., GPT-4o or Claude 3.5 Sonnet), ensuring superior contextual reasoning, deeper nuance in local recommendations, and near-instantaneous streaming

generation.

- **Advanced Customization:** Access to hyper-specific filter parameters beyond the standard 3x3 questionnaire (e.g., strict dietary requirements, wheelchair accessibility constraints).
- **Offline Export:** The ability to download generated itineraries and maps for fully offline GPS navigation.

7.3 B2B Revenue: Local Business Promotion

A critical revenue stream for Kemora relies on the B2B (Business-to-Business) promotion of local Egyptian restaurants, cafes, and hospitality venues.

7.3.1 Planned Algorithmic Bidding for Promoted Places

Because Kemora's backend explicitly injects Foursquare and localized database places into the AI's context window (via the RAG pipeline) before the itinerary is generated, there is an immense opportunity to monetize this injection mechanism. While the current implementation organically ranks and injects top-rated venues within a 30km radius, the architecture natively supports future B2B monetization. Local businesses could pay for "Featured" placement. Under such a model, when the backend constructs the context array, sponsored places matching the user's explicit budget and style preferences would be guaranteed injection into the prompt with a "highly recommended" flag. The LLM would then organically weave these sponsored locations into the narrative of the itinerary. This paradigm offers incredibly high-conversion, native advertising that avoids disrupting the user experience with traditional banner ads.

7.4 Socio-Economic Impact

Beyond software metrics, Kemora is designed to enact a tangible shift in the Egyptian tourism economy. By utilizing a comprehensive local database and Foursquare integration, the RAG pipeline is capable of surfacing a diverse mix of highly-rated entities

beyond just mainstream monoliths like the Pyramids of Giza, distributing tourist foot traffic more evenly.

This technological democratization empowers local artisans, boutique hotel owners in regions like Siwa and Nubia, and authentic neighborhood eateries in Cairo. By removing the friction of discovering these locations and automatically organizing them into logically routed daily plans, Kemora actively shifts tourist spending toward the local Egyptian economy, thereby maximizing the grassroots economic benefit of the tourism sector.

Chapter 8

Conclusion & Future Work

8.1 Summary of Achievements

Over the course of the 7-month project lifecycle, the development of Kemora has evolved from a conceptual blueprint into a robust, comprehensive case study in modern full-stack software engineering and applied artificial intelligence. The primary ambition of the project was to eradicate the fragmentation inherent in the Egyptian travel sector by providing a unified, intelligent, and socially driven discovery platform.

Through the rigorous application of the .NET 10 Clean Architecture paradigm, the backend was engineered to be highly decoupled, testable, and resilient. The integration of Entity Framework Core facilitated a robust relational data model capable of handling the core domain entities and intricate social interactions, though spatial filtering is currently performed in memory. On the frontend, Flutter provided the necessary cross-platform capabilities to implement the bespoke "Modern Heritage" design language, delivering a visually stunning, responsive, and consistent 60fps mobile experience across both iOS and Android platforms.

The most significant and innovative achievement of the project is the dynamic AI Trip Planning pipeline. By deliberately stepping away from monolithic, black-box AI dependencies and instead integrating with OpenRouter, the system achieved model-agnosticism, flexibility, and cost-efficiency. Furthermore, by implementing a bespoke Context-Construction logic—injecting real, highly-rated Google Maps data into the prompt prior to generation—the system effectively solved the pervasive issue of LLM hallucination in travel planning. The subsequent optimizations, such as transitioning from sequential bulk image fetching to targeted concurrent fetching, demonstrated a steadfast commitment to system performance, latency reduction, and end-user experience.

8.2 Limitations

While the platform is highly functional and meets its initial objectives, several limitations remain in the current iteration that prevent it from functioning at a massive enterprise scale:

- **Database Scalability:** The current implementation relies on a centralized SQL Server instance. While sufficient for local development and the targeted load benchmarks presented in Chapter 5, a monolithic relational database can become a bottleneck in a highly distributed, high-throughput production environment.
- **Background Processing:** Currently, background tasks (such as place data hydration, thumbnail generation, and email dispatch) are handled via simple in-memory `Task.Run` threads. This approach lacks durability and fault-tolerance; if the application crashes or recycles during execution, the background task is permanently lost.
- **Offline Support:** The Flutter application necessitates a persistent internet connection to function. It currently lacks a comprehensive offline-first architecture (e.g., Hive, SQLite caching, or an offline queuing system for the social feed), limiting accessibility in areas with poor cellular coverage.

8.3 Future Work

To transition Kemora from a highly polished academic prototype into a production-ready, commercial enterprise platform capable of serving millions of concurrent users, the following architectural and infrastructural enhancements are proposed.

8.3.1 Microservices Architecture and Kubernetes Deployment

As user traffic and feature complexity grow, the current monolithic Clean Architecture backend should be strategically decomposed into a distributed Microservices Architecture. Domain boundaries would dictate the separation of concerns into distinct autonomous services, such as an Identity Service, a Trip Planning Service, and a Social Interaction Service. To address the aforementioned database scalability limitations, each microservice

would manage its own isolated database (the database-per-service pattern), ensuring loose coupling and scalable data persistence.

To orchestrate these services, a migration to Kubernetes (K8s) is essential. Kubernetes would provide the backbone for container orchestration, offering theoretical implementations of auto-scaling, self-healing pods, and zero-downtime rolling deployments. Each microservice would be encapsulated within its own Docker container and managed via Kubernetes Deployments and Services. An Ingress Controller would act as the API Gateway, dynamically routing external requests to the appropriate internal services, while horizontal pod autoscalers (HPA) would automatically provision more instances of the Trip Planning Service during periods of high demand.

8.3.2 Event-Driven Architecture via RabbitMQ

To address the limitations of in-memory background processing and to further decouple the microservices, the integration of a robust message broker such as RabbitMQ is critical. Implementing an event-driven architecture using RabbitMQ would ensure durable, asynchronous background processing.

Heavy, time-consuming workloads—such as concurrent external API fetching from Google Places, asynchronous image processing, or sending transactional emails—would be published as discrete messages to a RabbitMQ queue. Dedicated worker nodes would then consume and process these messages independently of the main web API threads. This ensures that no data or tasks are lost in the event of a node failure, as unacknowledged messages would simply be re-queued. Furthermore, a publish/subscribe (Pub/Sub) pattern would enable services to react to domain events (e.g., `UserRegisteredEvent` triggering a welcome email) without tight coupling.

8.3.3 Native Open-Source LLM Fine-Tuning

Currently, the AI pipeline relies on OpenRouter to interact with third-party foundational models. While cost-effective during development, a commercial-scale platform necessitates complete control over its AI infrastructure, data privacy, and latency overhead. Future iterations should focus on hosting and fine-tuning an open-source Large Language Model (such as LLaMA 3 or Mistral) natively on dedicated GPU clusters or cloud instances (e.g., AWS EC2 P4 or Azure ND-series).

By utilizing parameter-efficient fine-tuning techniques like Low-Rank Adaptation (LoRA) or QLoRA, the open-source model could be explicitly trained on Egyptian travel datasets, local dialects, and historical context. Furthermore, natively hosting the model eliminates external API latency and strict rate limits, paving the way for real-time, streaming AI responses and a completely sovereign AI architecture with no third-party dependencies.

8.3.4 Advanced AI Personalization via Vector Databases

The AI pipeline can be drastically enhanced by incorporating Vector Databases (e.g., Pinecone, Milvus, or Qdrant) and text embeddings. Instead of merely injecting places within a fixed geographic radius into the prompt, the system could utilize Retrieval-Augmented Generation (RAG) driven by semantic search. By converting user preferences and historical interaction data into vector embeddings, the system can perform a similarity search to find locations that perfectly match a user's deeply nuanced implicit desires (e.g., "quiet bohemian cafes with strong Wi-Fi and vegan options suitable for remote work") before injecting those specific locations into the LLM context window.

8.3.5 Offline-First Mobile Architecture

To resolve the dependency on a persistent internet connection, the Flutter frontend must transition to an offline-first architecture. By integrating local persistence solutions such as SQLite or Hive, the application can aggressively cache trip itineraries, map tiles, and social feeds. Furthermore, implementing an offline queuing system would allow users to create posts or react to trips while disconnected, automatically synchronizing with the backend once cellular connectivity is restored.

8.3.6 Comprehensive Arabic Localization

To maximize user adoption and penetration within the domestic Egyptian market and the broader MENA region, comprehensive Right-To-Left (RTL) support and complete Arabic localization must be implemented. This extends beyond UI translation in Flutter; it requires localized Google Maps queries and prompt engineering specifically designed to yield high-quality, culturally resonant Arabic itineraries from the LLM.

8.4 Concluding Remarks

The Kemora project successfully demonstrates the immense potential of fusing cutting-edge Generative AI with traditional, rigorous software engineering best practices. Over the course of 7 months, it has established a solid, extensible foundation that not only solves immediate user pain points in travel discovery but is architecturally poised to scale. By embracing microservices, distributed messaging, and native AI fine-tuning in the future, Kemora has the potential to evolve into a dominant, ubiquitous platform within the regional TravelTech sector.

Bibliography

[1] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[2] Google. *Flutter Architectural Overview*. `https://docs.flutter.dev/resources/architectural-overview`. Accessed June 2026.

[3] OpenRouter. *OpenRouter API Documentation*. `https://openrouter.ai/docs`. Accessed June 2026.

[4] Microsoft. *Entity Framework Core Documentation*. `https://learn.microsoft.com/en-us/ef/core/`. Accessed June 2026.

[5] Gene M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities." *AFIPS Conference Proceedings*, vol. 30, pp. 483-485, 1967.

[6] R. W. Sinnott. "Virtues of the Haversine." *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[7] Foursquare. *Foursquare Places API Documentation*. `https://location.foursquare.com/developer/reference/places-api-overview`. Accessed June 2026.

Appendix A

API Documentation Reference

A.1 Overview

This appendix provides a comprehensive reference for the Kemora RESTful API. The API is structured around REST principles, utilizing standard HTTP methods and returning JSON formatted responses. Authentication is enforced via JWT Bearer tokens.

A.2 Auth Endpoints

A.2.1 POST `/api/v1/auth/register`

Register a new user account.

- **Authorization:** None (Public)
- **Request Body:** `RegisterDto`
- **Response (200):** Returns `AuthResponseDto`
- **Response (400):** Empty/Void

A.2.2 POST `/api/v1/auth/login`

Login with email and password.

- **Authorization:** None (Public)
- **Request Body:** `LoginDto`
- **Response (200):** Returns `AuthResponseDto`
- **Response (401):** Empty/Void

A.2.3 POST `/api/v1/auth/google-login`

Login using Google OAuth token.

- **Authorization:** None (Public)
- **Request Body:** `GoogleLoginDto`
- **Response (200):** Returns `AuthResponseDto`
- **Response (401):** Empty/Void
- **Response (400):** Empty/Void

A.2.4 POST `/api/v1/auth/refresh`

Refresh an expired JWT token using a valid refresh token.

- **Authorization:** Bearer Token Required
- **Request Body:** `RefreshTokenRequestDto`
- **Response (200):** Returns `AuthResponseDto`
- **Response (401):** Empty/Void

A.2.5 POST `/api/v1/auth/confirm-email`

Confirm a user's email address using the confirmation token sent during registration.

- **Authorization:** None (Public)
- **Request Body:** `ConfirmEmailDto`
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.2.6 GET `/api/v1/auth/confirm-email-link`

Confirm email via a direct link (GET request).

- **Authorization:** None (Public)
- **Query Parameters:** `userId (string)`, `token (string)`

A.2.7 POST /api/v1/auth/forgot-password

Request a password reset token. An email will be sent if the account exists.

- **Authorization:** None (Public)
- **Request Body:** `ForgotPasswordDto`
- **Response (200):** Empty/Void

A.2.8 GET /api/v1/auth/reset-password-redirect

Redirects the user to the Flutter app's password reset screen.

- **Authorization:** None (Public)
- **Query Parameters:** `email (string)`, `token (string)`

A.2.9 POST /api/v1/auth/reset-password

Reset a user's password using the token received via email.

- **Authorization:** Bearer Token Required
- **Request Body:** `ResetPasswordDto`
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.2.10 POST /api/v1/auth/admin/send-confirmation

Manually trigger an email confirmation for a user (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Query Parameters:** `email (string)`
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.2.11 POST /api/v1/auth/preferences

- **Authorization:** Bearer Token Required
- **Request Body:** `JsonElement`

A.2.12 POST /api/v1/auth/change-password

Change the authenticated user's password.

- **Authorization:** Bearer Token Required
- **Request Body:** `ChangePasswordDto`
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.2.13 POST /api/v1/auth/change-email

Change the authenticated user's email address.

- **Authorization:** Bearer Token Required
- **Request Body:** `ChangeEmailDto`
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.3 Badges Endpoints

A.3.1 POST /api/v1/admin/badges

Create a new badge (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateBadgeDto`
- **Response (200):** Returns `BadgeResponseDto`

A.3.2 GET `/api/v1/badges`

Get all available badges.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<BadgeResponseDto>`

A.3.3 GET `/api/v1/my/badges`

Get the authenticated user's earned badges.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `List<UserBadgeResponseDto>`

A.3.4 GET `/api/v1/my/points`

Get the authenticated user's point summary.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `PointsSummaryDto`

A.3.5 GET `/api/v1/leaderboard`

Get the top users by points leaderboard.

- **Authorization:** None (Public)
- **Query Parameters:** `top (int)`
- **Response (200):** Returns `List<LeaderboardEntryDto>`

A.4 Chats Endpoints

A.4.1 POST `/api/v1/chats/send`

- **Authorization:** Bearer Token Required
- **Request Body:** `SendMessageDto`

A.4.2 GET /api/v1/chats/conversations

- **Authorization:** Bearer Token Required

A.4.3 GET /api/v1/chats/conversation/\contactId\

- **Authorization:** Bearer Token Required
- **Query Parameters:** `page (int)`, `pageSize (int)`

A.4.4 POST /api/v1/chats/read/\contactId\

- **Authorization:** Bearer Token Required

A.5 Comments Endpoints

A.5.1 POST /api/v1/posts/\postId\/comments

Add a comment to a post.

- **Authorization:** Bearer Token Required
- **Request Body:** `CreateCommentDto`
- **Response (200):** Returns `CommentResponseDto`
- **Response (404):** Empty/Void

A.5.2 GET /api/v1/posts/\postId\/comments

Get paginated comments for a post.

- **Authorization:** None (Public)
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<CommentResponseDto>`

A.5.3 PUT /api/v1/comments/\id\

Update your own comment.

- **Authorization:** Bearer Token Required
- **Request Body:** `UpdateCommentDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.5.4 DELETE /api/v1/comments/\id\

Delete your own comment.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.6 Events Endpoints

A.6.1 POST /api/v1/places/\placeId\/events

Create an event for a specific place (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateEventDto`
- **Response (200):** Returns `EventResponseDto`
- **Response (404):** Empty/Void

A.6.2 GET /api/v1/events

Get upcoming events across all places.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<EventResponseDto>`

A.6.3 GET /api/v1/places/\placeId\/events

Get events for a specific place.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<EventResponseDto>`

A.6.4 PUT /api/v1/events/\id\

Update an existing event (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `UpdateEventDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.6.5 DELETE /api/v1/events/\id\

Delete an event (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.7 Favorites Endpoints

A.7.1 POST /api/v1/favorites/\placeId\

Add a place to favorites.

- **Authorization:** Bearer Token Required
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.7.2 DELETE `/api/v1/favorites/\placeId\`

Remove a place from favorites.

- **Authorization:** Bearer Token Required
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.7.3 GET `/api/v1/favorites`

Get all favorite places for the authenticated user.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `List<PlacePublicDto>`

A.7.4 GET `/api/v1/favorites/\placeId\/check`

Check if a specific place is in the user's favorites.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `FavoriteCheckDto`

A.8 Images Endpoints

A.8.1 POST `/api/v1/images/upload`

Upload an image file (Admin only). Returns the public URL of the uploaded image.

- **Authorization:** Bearer Token Required
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void
- **Response (500):** Empty/Void

A.9 Notifications Endpoints

A.9.1 GET `/api/v1/notifications`

Get paginated notifications for the authenticated user.

- **Authorization:** Bearer Token Required
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<NotificationDto>`

A.9.2 GET `/api/v1/notifications/unread-count`

Get the count of unread notifications.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `int`

A.9.3 PUT `/api/v1/notifications/\id\/read`

Mark a specific notification as read.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void

A.9.4 PUT `/api/v1/notifications/read-all`

Mark all notifications as read.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void

A.10 Photos Endpoints

A.10.1 POST /api/v1/places/\placeId\/photos

Add a photo to a place (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreatePhotoDto`
- **Response (200):** Returns `PhotoResponseDto`
- **Response (404):** Empty/Void

A.10.2 GET /api/v1/places/\placeId\/photos

Get all photos for a place.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<PhotoResponseDto>`

A.10.3 PUT /api/v1/places/\placeId\/photos/\photoId\/main

Set a photo as the main photo for a place (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.10.4 DELETE /api/v1/places/\placeId\/photos/\photoId\

Delete a photo from a place (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.10.5 GET /api/v1/proxy

Proxies a photo from Google Places API to avoid exposing the API key.

- **Authorization:** None (Public)
- **Query Parameters:** `name (string)`
- **Response (200):** Returns `FileResult`

A.11 Places Endpoints

A.11.1 GET /api/v1/places

Browse places with optional filters. Supports search, category, and governorate filtering.

- **Authorization:** None (Public)
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<PlacePublicDto>`

A.11.2 GET /api/v1/places/\id\

Get detailed information about a specific place including photos, reviews, and events.

- **Authorization:** None (Public)
- **Response (200):** Returns `PlaceDetailPublicDto`
- **Response (404):** Empty/Void

A.11.3 POST /api/v1/places/trip-plan

Generate an AI-powered trip plan based on location, budget, and preferences.

- **Authorization:** None (Public)
- **Request Body:** `TripPlanRequestDto`
- **Response (200):** Returns `TripPlanResponseDto`
- **Response (400):** Empty/Void

A.11.4 GET /api/v1/places/governorates

Get all governorates in Egypt.

- **Authorization:** None (Public)

A.11.5 GET /api/v1/places/top

Get the top 20 featured places across Egypt.

- **Authorization:** None (Public)

A.11.6 GET /api/v1/places/swap

Request an alternative for a specific place in a trip plan.

- **Authorization:** None (Public)
- **Query Parameters:** `currentPlaceName (string)`, `preferences (string)`

A.12 PlacesManagement Endpoints

A.12.1 POST /api/v1/admin/governorates

Create a new governorate.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateGovernorateDto`
- **Response (200):** Returns `GovernorateDto`

A.12.2 GET /api/v1/admin/governorates

Get all governorates.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<GovernorateDto>`

A.12.3 PUT `/api/v1/admin/governorates/\id\`

Update a governorate.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateGovernorateDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.4 DELETE `/api/v1/admin/governorates/\id\`

Delete a governorate.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.5 POST `/api/v1/admin/categories`

Create a new category.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateCategoryDto`
- **Response (200):** Returns `CategoryDto`

A.12.6 GET `/api/v1/admin/categories`

Get all categories.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<CategoryDto>`

A.12.7 PUT /api/v1/admin/categories/\id\

Update a category.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreateCategoryDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.8 DELETE /api/v1/admin/categories/\id\

Delete a category.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.9 POST /api/v1/admin/placetypes

Create a new place type.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreatePlaceTypeDto`
- **Response (200):** Returns `PlaceTypeDto`
- **Response (400):** Empty/Void

A.12.10 GET /api/v1/admin/placetypes

Get all place types.

- **Authorization:** None (Public)
- **Response (200):** Returns `List<PlaceTypeDto>`

A.12.11 PUT /api/v1/admin/placetypes/\id\

Update a place type.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreatePlaceTypeDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.12 DELETE /api/v1/admin/placetypes/\id\

Delete a place type.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.13 POST /api/v1/admin/places

Create a new place.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `CreatePlaceDto`
- **Response (200):** Returns `PlaceAdminDto`
- **Response (400):** Empty/Void

A.12.14 PUT /api/v1/admin/places/\id\

Update a place.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Request Body:** `UpdatePlaceDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.12.15 DELETE /api/v1/admin/places/\id\

Delete a place.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.13 Posts Endpoints

A.13.1 POST /api/v1/posts

Create a new community post.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `PostListResponseDto`

A.13.2 POST /api/v1/posts/image

Upload an image for a post and get its URL.

- **Authorization:** Bearer Token Required
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.13.3 GET /api/v1/posts

Browse all posts with pagination.

- **Authorization:** None (Public)
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<PostListResponseDto>`

A.13.4 GET /api/v1/posts/\id\

Get a single post with full details.

- **Authorization:** None (Public)
- **Response (200):** Returns `PostDetailResponseDto`
- **Response (404):** Empty/Void

A.13.5 GET /api/v1/posts/my

Get the authenticated user's posts.

- **Authorization:** Bearer Token Required
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<PostListResponseDto>`

A.13.6 POST /api/v1/posts/\id\/like

Toggle a like/reaction on a post.

- **Authorization:** Bearer Token Required

A.13.7 POST /api/v1/posts/\id\/comment

Add a comment to a post.

- **Authorization:** Bearer Token Required
- **Request Body:** `CreateCommentDto`

A.13.8 PUT /api/v1/posts/\id\

Update your own post.

- **Authorization:** Bearer Token Required
- **Request Body:** `UpdatePostDto`

- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.13.9 DELETE `/api/v1/posts/\id\`

Delete your own post.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.14 Profile Endpoints

A.14.1 GET `/api/v1/profile/my`

Get the authenticated user's profile.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `ProfileDto`
- **Response (404):** Empty/Void

A.14.2 PUT `/api/v1/profile/my`

Update the authenticated user's profile.

- **Authorization:** Bearer Token Required
- **Request Body:** `UpdateProfileDto`
- **Response (204):** Empty/Void
- **Response (400):** Empty/Void

A.14.3 GET /api/v1/profile/\id\/public

Get a user's public profile by user ID.

- **Authorization:** None (Public)
- **Response (200):** Returns `PublicProfileDto`
- **Response (404):** Empty/Void

A.14.4 POST /api/v1/profile/image

Upload a new profile picture.

- **Authorization:** Bearer Token Required
- **Response (200):** Empty/Void
- **Response (400):** Empty/Void

A.15 Reactions Endpoints

A.15.1 POST /api/v1/posts/\postId\/react

Add or remove a reaction on a post.

- **Authorization:** Bearer Token Required
- **Request Body:** `ReactToPostDto`
- **Response (204):** Empty/Void
- **Response (400):** Empty/Void

A.15.2 POST /api/v1/comments/\commentId\/react

Add or remove a reaction on a comment.

- **Authorization:** Bearer Token Required
- **Request Body:** `ReactToCommentDto`

- **Response (204):** Empty/Void
- **Response (400):** Empty/Void

A.16 Reviews Endpoints

A.16.1 POST `/api/v1/places/\placeId\/reviews`

Submit a review for a place.

- **Authorization:** Bearer Token Required
- **Request Body:** `CreateReviewDto`
- **Response (200):** Returns `ReviewResponseDto`
- **Response (404):** Empty/Void

A.16.2 GET `/api/v1/places/\placeId\/reviews`

Get paginated reviews for a place.

- **Authorization:** None (Public)
- **Query Parameters:** `page (int)`, `pageSize (int)`
- **Response (200):** Returns `PagedResult<ReviewResponseDto>`

A.16.3 DELETE `/api/v1/places/\placeId\/reviews/\id\`

Delete your own review.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.17 Stories Endpoints

A.17.1 POST /api/v1/stories

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `StoryResponseDto`

A.17.2 GET /api/v1/stories

- **Authorization:** None (Public)
- **Response (200):** Returns `IEnumerable<object>`

A.17.3 GET /api/v1/stories/\userId\

- **Authorization:** None (Public)
- **Response (200):** Returns `IEnumerable<StoryResponseDto>`

A.17.4 DELETE /api/v1/stories/\id\

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.18 Trips Endpoints

A.18.1 POST /api/v1/trips

Create a new trip.

- **Authorization:** Bearer Token Required
- **Request Body:** `CreateTripDto`
- **Response (201):** Returns `TripDetailDto`
- **Response (400):** Empty/Void

A.18.2 GET /api/v1/trips

List the authenticated user's trips with pagination.

- **Authorization:** Bearer Token Required
- **Query Parameters:** `page (int)`, `ps (int)`
- **Response (200):** Returns `PagedResult<TripListDto>`

A.18.3 GET /api/v1/trips/\id\

Get a specific trip with its places.

- **Authorization:** Bearer Token Required
- **Response (200):** Returns `TripDetailDto`
- **Response (404):** Empty/Void

A.18.4 PUT /api/v1/trips/\id\

Update a trip's details.

- **Authorization:** Bearer Token Required
- **Request Body:** `UpdateTripDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void
- **Response (400):** Empty/Void

A.18.5 DELETE /api/v1/trips/\id\

Delete a trip.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.18.6 POST /api/v1/trips/\id\/places

Add a place to a trip's itinerary.

- **Authorization:** Bearer Token Required
- **Request Body:** `AddTripPlaceDto`
- **Response (200):** Returns `TripPlaceResponseDto`
- **Response (400):** Empty/Void

A.18.7 PUT /api/v1/trips/\tripId\/places/\tpId\

Update a place's order or notes in the trip itinerary.

- **Authorization:** Bearer Token Required
- **Request Body:** `UpdateTripPlaceDto`
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.18.8 DELETE /api/v1/trips/\tripId\/places/\tpId\

Remove a place from the trip itinerary.

- **Authorization:** Bearer Token Required
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.18.9 POST /api/v1/trips/save-plan

Save a complete AI-generated trip plan. Requires authentication.

- **Authorization:** Bearer Token Required
- **Request Body:** `SaveAIPlanDto`
- **Response (201):** Returns `TripDetailDto`
- **Response (401):** Empty/Void

A.19 UserManagement Endpoints

A.19.1 GET `/api/v1/admin/users`

Get all registered users (Admin only).

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (200):** Returns `List<ProfileDto>`

A.19.2 POST `/api/v1/admin/users/\userId\/roles/\role\`

Assign a role to a user (e.g., "Admin", "User").

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

A.19.3 DELETE `/api/v1/admin/users/\userId\/roles/\role\`

Remove a role from a user.

- **Authorization:** Bearer Token Required (Roles: Admin)
- **Response (204):** Empty/Void
- **Response (404):** Empty/Void

Appendix B

Critical Source Code Implementations

B.1 Overview

This appendix provides the core implementation of the `OpenRouterAiService`, which is responsible for the dynamic orchestration of Large Language Models within the Kemora backend. This service highlights the robust error handling, rate-limit parsing, and dynamic model selection mechanisms discussed in Chapter 4.

B.2 OpenRouterAiService.cs Implementation

```
using System.Collections.Concurrent;
using System.Net.Http.Json;
using System.Text.Json.Serialization;
using Kemora.Domain.Interfaces;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;

namespace Kemora.Infrastructure.Services
{
    public class OpenRouterAiService : IAiService
    {
        private readonly IHttpClientFactory _httpFactory;
        private readonly IConfiguration _config;
        private readonly ILogger<OpenRouterAiService> _logger;

        private class ModelInfo
        {
            public string Id { get; set; } = string.Empty;
            public long ContextLength { get; set; }
            public int RequestCount { get; set; }
            public DateTime WindowStart { get; set; }
            public bool IsExhausted { get; set; }
```

```
23              public DateTime? ResetTime { get; set; }
24          }
25
26          private readonly ConcurrentDictionary<string, ModelInfo>
    _models = new();
27          private readonly SemaphoreSlim _initSemaphore = new(1, 1);
28          private bool _isInitialized = false;
29
30          public OpenRouterAiService(
31              IHttpClientFactory httpFactory,
32              IConfiguration config,
33              ILogger<OpenRouterAiService> logger)
34          {
35              _httpFactory = httpFactory;
36              _config = config;
37              _logger = logger;
38          }
39
40          private string GetApiKey() => _config["OpenRouter:ApiKey"] ??
    string.Empty;
41          private int MaxRequestsPerModel() => _config.GetValue<int>("
    OpenRouter:MaxRequestsPerModelPerDay", 40);
42
43          private async Task EnsureInitializedAsync()
44          {
45              if (_isInitialized) return;
46
47              await _initSemaphore.WaitAsync();
48              try
49              {
50                  if (_isInitialized) return;
51
52                  var client = _httpFactory.CreateClient("OpenRouter");
53                  var response = await client.GetAsync("https://
    openrouter.ai/api/v1/models");
54
55                  if (response.IsSuccessStatusCode)
56                  {
```

```
57                  var result = await response.Content.
    ReadFromJsonAsync<OpenRouterModelsResponse>();
58                  if (result?.Data != null)
59                  {
60                      var now = DateTime.UtcNow;
61                      foreach (var model in result.Data)
62                      {
63                          // Filter free models with 0 prompt and 0
    completion pricing
64                          if (model.Pricing != null &&
65                              double.TryParse(model.Pricing.Prompt,
    out var promptPrice) && promptPrice == 0 &&
66                              double.TryParse(model.Pricing.
    Completion, out var compPrice) && compPrice == 0)
67                          {
68                              // Exclude known paid/broken models
    that falsely report $0 pricing
69                              var blocklist = new[] { "lyria", "
    imagen", "owl-alpha", "ocr-fast", "music" };
70                              if (model.ContextLength >= 8192 &&
71                                  !blocklist.Any(b => model.Id.
    Contains(b, StringComparison.OrdinalIgnoreCase)))
72                              {
73                                  _models[model.Id] = new ModelInfo
74                                  {
75                                      Id = model.Id,
76                                      ContextLength = model.
    ContextLength,
77                                      RequestCount = 0,
78                                      WindowStart = now,
79                                      IsExhausted = false
80                                  };
81                              }
82                          }
83                      }
84                      _logger.LogInformation("Loaded {Count} free
    models from OpenRouter", _models.Count);
85                  }
86              }
```

```
87                else
88                {
89                    _logger.LogWarning("Failed to fetch OpenRouter
    models. Code: {StatusCode}", response.StatusCode);
90                }
91
92                _isInitialized = true;
93            }
94            catch (Exception ex)
95            {
96                _logger.LogError(ex, "Exception while initializing
    OpenRouter models.");
97            }
98            finally
99            {
100               _initSemaphore.Release();
101           }
102       }
103
104       private string SelectBestModel(int estimatedTokens, IEnumerable
    <string>? excludeModels = null)
105       {
106           var now = DateTime.UtcNow;
107           var maxReqs = MaxRequestsPerModel();
108           var excludeSet = excludeModels?.ToHashSet() ?? new HashSet<
    string>();
109
110           foreach (var prop in _models.Values)
111           {
112               // Reset exhausted status if ResetTime has passed
113               if (prop.IsExhausted && prop.ResetTime.HasValue && now
    >= prop.ResetTime.Value)
114               {
115                   prop.IsExhausted = false;
116                   prop.ResetTime = null;
117                   prop.RequestCount = 0;
118                   prop.WindowStart = now;
119               }
```

```
120                 else if (!prop.IsExhausted && now.Subtract(prop.
    WindowStart).TotalHours >= 24)
121                 {
122                     prop.RequestCount = 0;
123                     prop.WindowStart = now;
124                 }
125             }
126 
127             var availableModels = _models.Values
128                 .Where(m => !m.IsExhausted && m.RequestCount < maxReqs
    && !excludeSet.Contains(m.Id))
129                 .Where(m => m.ContextLength >= estimatedTokens * 1.5)
    // Buffer for output and system prompts
130                 .OrderBy(m => m.RequestCount)
131                 .ThenByDescending(m => m.ContextLength)
132                 .ToList();
133 
134             if (availableModels.Any())
135             {
136                 var selected = availableModels.First();
137                 selected.RequestCount++;
138                 return selected.Id;
139             }
140 
141             return "openrouter/free";
142         }
143 
144         private void HandleRateLimitHeaders(System.Net.Http.
    HttpResponseMessage response, string modelId)
145         {
146             if (!_models.TryGetValue(modelId, out var modelInfo))
    return;
147 
148             var headers = response.Headers;
149             DateTime? resetTime = null;
150 
151             // Try standard rate limit headers
152             if (headers.TryGetValues("x-ratelimit-reset", out var
    resetValues) || headers.TryGetValues("x-ratelimit-reset-requests",
```

```
out resetValues))
153             {
154                 var resetStr = resetValues.FirstOrDefault();
155                 if (!string.IsNullOrEmpty(resetStr))
156                 {
157                     // OpenRouter might send a string like "10s", "1m",
 "24h" or a unix timestamp
158                     if (resetStr.EndsWith("s") && double.TryParse(
resetStr.TrimEnd('s'), out var secs))
159                     {
160                         resetTime = DateTime.UtcNow.AddSeconds(secs);
161                     }
162                     else if (resetStr.EndsWith("m") && double.TryParse(
resetStr.TrimEnd('m'), out var mins))
163                     {
164                         resetTime = DateTime.UtcNow.AddMinutes(mins);
165                     }
166                     else if (resetStr.EndsWith("h") && double.TryParse(
resetStr.TrimEnd('h'), out var hrs))
167                     {
168                         resetTime = DateTime.UtcNow.AddHours(hrs);
169                     }
170                     else if (long.TryParse(resetStr, out var timestamp)
)
171                     {
172                         // Could be seconds or milliseconds
173                         if (timestamp > 2000000000000) // Milliseconds
174                         {
175                             resetTime = DateTimeOffset.
FromUnixTimeMilliseconds(timestamp).UtcDateTime;
176                         }
177                         else
178                         {
179                             resetTime = DateTimeOffset.
FromUnixTimeSeconds(timestamp).UtcDateTime;
180                         }
181                     }
182                 }
183             }
```

```
184
185             if (resetTime.HasValue)
186             {
187                 modelInfo.ResetTime = resetTime;
188                 modelInfo.IsExhausted = true;
189                 _logger.LogWarning("Model {Model} rate limited. Reset
    time set to {ResetTime} UTC.", modelId, resetTime);
190             }
191             else
192             {
193                 // Fallback to 1 hour if no header
194                 modelInfo.ResetTime = DateTime.UtcNow.AddHours(1);
195                 modelInfo.IsExhausted = true;
196             }
197         }
198
199         public async Task<string> GenerateCompletionAsync(string
    systemPrompt, string userPrompt, double temperature = 0.3, bool
    jsonMode = true)
200         {
201             var apiKey = GetApiKey();
202             if (string.IsNullOrEmpty(apiKey))
203             {
204                 _logger.LogError("OpenRouter:ApiKey is missing.");
205                 return "AI Error: Missing API Key.";
206             }
207
208             await EnsureInitializedAsync();
209
210             // Very rough token estimation (1 token approx 4 chars)
211             int estimatedTokens = (systemPrompt.Length + userPrompt.
    Length) / 4;
212
213             int attempts = 0;
214             int maxAttempts = Math.Max(3, _models.Count > 0 ? _models.
    Count : 3);
215             var triedModels = new HashSet<string>();
216
217             while (attempts < maxAttempts)
```

```
218             {
219                 attempts++;
220                 var selectedModel = SelectBestModel(estimatedTokens,
    triedModels);
221                 triedModels.Add(selectedModel);
222 
223                 var fallbackModels = _models.Values.Where(m => m.Id !=
    selectedModel && !triedModels.Contains(m.Id)).Select(m => m.Id).Take
    (2).ToList();
224 
225                 var requestBody = new
226                 {
227                     model = selectedModel,
228                     messages = new[]
229                     {
230                         new { role = "system", content = systemPrompt
    },
231                         new { role = "user", content = userPrompt }
232                     },
233                     temperature = temperature,
234                     response_format = jsonMode ? new { type = "
    json_object" } : null,
235                     models = fallbackModels,
236                     route = "fallback"
237                 };
238 
239                 var client = _httpFactory.CreateClient("OpenRouter");
240                 client.DefaultRequestHeaders.Authorization = new System
    .Net.Http.Headers.AuthenticationHeaderValue("Bearer", apiKey);
241                 client.DefaultRequestHeaders.Add("HTTP-Referer", "https
    ://kemora.app"); // Good practice for OpenRouter
242                 client.DefaultRequestHeaders.Add("X-Title", "Kemora AI
    Planner");
243 
244                 try
245                 {
246                     var response = await client.PostAsJsonAsync("https
    ://openrouter.ai/api/v1/chat/completions", requestBody);
247 
```

```
248                     if (!response.IsSuccessStatusCode)
249                     {
250                         var statusCode = response.StatusCode;
251                         var errorContent = await response.Content.
    ReadAsStringAsync();
252                         _logger.LogWarning("OpenRouter API error for
    model {Model} ({Code}): {Error}", selectedModel, statusCode,
    errorContent);
253 
254                         // If rate limited or not found
255                         if (statusCode == System.Net.HttpStatusCode.
    TooManyRequests)
256                         {
257                             HandleRateLimitHeaders(response,
    selectedModel);
258                             continue;
259                         }
260 
261                         if (statusCode == System.Net.HttpStatusCode.
    NotFound)
262                         {
263                             if (_models.TryGetValue(selectedModel, out
    var info))
264                             {
265                                 info.IsExhausted = true;
266                                 info.ResetTime = DateTime.UtcNow.
    AddDays(1); // 404 means model is likely gone
267                             }
268                             continue;
269                         }
270 
271                         // 402 Payment Required     model is NOT
    actually free, remove permanently
272                         if (statusCode == System.Net.HttpStatusCode.
    PaymentRequired)
273                         {
274                             _logger.LogWarning("Model {Model} requires
    payment     removing from free pool permanently.", selectedModel);
275                             _models.TryRemove(selectedModel, out _);
```

```
                continue;
            }

            if ((int)statusCode >= 500)
            {
                // Transient server error, try next
                continue;
            }

            // For 400 Bad Request (e.g. context too big), we can also try next if there are models with larger context
            if (statusCode == System.Net.HttpStatusCode.BadRequest)
            {
                 continue;
            }

            return $"AI Error: {statusCode}";
        }

        var result = await response.Content.ReadFromJsonAsync<OpenRouterResponse>();
        var content = result?.Choices?.FirstOrDefault()?.Message?.Content;

        if (string.IsNullOrEmpty(content))
        {
            _logger.LogWarning("AI returned empty response for model {Model}. Marking as exhausted and retrying...", selectedModel);
            if (_models.TryGetValue(selectedModel, out var info))
            {
                info.IsExhausted = true;
                info.ResetTime = DateTime.UtcNow.AddMinutes(5);
            }
            continue;
        }
```

```
307
308                     if (jsonMode)
309                     {
310                         var trimmed = content.Trim();
311                         if (!(trimmed.StartsWith("{") || trimmed.
    StartsWith("[")))
312                         {
313                             _logger.LogWarning("AI returned non-JSON
    content in JSON mode for model {Model}. Attempt {Attempt}",
    selectedModel, attempts);
314                             continue;
315                         }
316                     }
317
318                     return content;
319                 }
320                 catch (Exception ex)
321                 {
322                     _logger.LogError(ex, "Failed to call OpenRouter API
     for model {Model}", selectedModel);
323                     if (attempts >= maxAttempts) return $"AI Error: {ex
    .Message}";
324                 }
325             }
326
327             return "AI Error: All free models are currently rate-
    limited or unavailable. Please try again later.";
328         }
329
330         private class OpenRouterModelsResponse { [JsonPropertyName("
    data")] public List<OpenRouterModel>? Data { get; set; } }
331         private class OpenRouterModel {
332             [JsonPropertyName("id")] public string Id { get; set; } =
    string.Empty;
333             [JsonPropertyName("context_length")] public long
    ContextLength { get; set; }
334             [JsonPropertyName("pricing")] public OpenRouterPricing?
    Pricing { get; set; }
335         }
```

```
336         private class OpenRouterPricing {
337             [JsonPropertyName("prompt")] public string Prompt { get;
    set; } = string.Empty;
338             [JsonPropertyName("completion")] public string Completion {
     get; set; } = string.Empty;
339         }
340         private class OpenRouterResponse { [JsonPropertyName("choices")
    ] public List<OpenRouterChoice>? Choices { get; set; } }
341         private class OpenRouterChoice { [JsonPropertyName("message")]
    public OpenRouterMessage? Message { get; set; } }
342         private class OpenRouterMessage { [JsonPropertyName("content")]
     public string? Content { get; set; } }
343     }
344 }
```

B.3 TripPlannerService.cs Implementation

This class is responsible for the core trip generation algorithm. It integrates geographic radius-based fetching, database caching mechanisms, and constructs dynamic itineraries by classifying places (e.g., hotels, restaurants, attractions) before dispatching contextually rich prompts to the AI service. The integration of local memory, Redis, and relational database persistence emphasizes the high-performance constraints detailed in the thesis.

```
1  using Kemora.Application.DTOs;
2  using Kemora.Application.Interfaces;
3  using Kemora.Domain.Entities;
4  using Kemora.Domain.Interfaces;
5  using Kemora.Domain.Models;
6  using Microsoft.Extensions.Logging;
7  using System;
8  using System.Linq;
9  using System.Threading.Tasks;
10 using System.Collections.Generic;
11
12 namespace Kemora.Application.Services
13 {
14     public class TripPlannerService : ITripPlannerService
15     {
```

```
16        private readonly IPlacesDataService _placesDataService;
17        private readonly IAiService _aiService;
18        private readonly IUnitOfWork _unitOfWork;
19        private readonly ICacheService _cache;
20        private readonly ILogger<TripPlannerService> _logger;
21
22        public TripPlannerService(
23            IPlacesDataService placesDataService,
24            IAiService aiService,
25            IUnitOfWork unitOfWork,
26            ICacheService cache,
27            ILogger<TripPlannerService> logger)
28        {
29            _placesDataService = placesDataService;
30            _aiService = aiService;
31            _unitOfWork = unitOfWork;
32            _cache = cache;
33            _logger = logger;
34        }
35
36        public async Task<TripPlanResponseDto> GenerateTripPlanAsync(
    TripPlanRequestDto request)
37        {
38            if (request.MinRadiusKm >= request.MaxRadiusKm)
39                throw new ArgumentException("MinRadiusKm must be less
    than MaxRadiusKm.");
40
41            // 1. Redis / Memory Cache handling
42            string prefs = request.Preferences ?? "none";
43            string cacheKey = $"plan_{request.CenterPlaceId}_{request.
    Latitude}_{request.Longitude}_{request.DurationDays}_{request.
    Location}_{prefs}_{request.AlternativeIndex}";
44
45            var cachedPlan = _cache.Get<TripPlanResponseDto>(cacheKey);
46            if (cachedPlan != null)
47                return cachedPlan;
48
49            // 1.5. DB Persistence Cache
```

```
50          var dbCachedPlan = await _unitOfWork.Repository<
    PrecomputedTripPlan>()
51              .FirstOrDefaultAsync(p => p.CacheKey == cacheKey);
52
53          if (dbCachedPlan != null)
54          {
55              var responseFromDb = new TripPlanResponseDto
56              {
57                  TripPlan = dbCachedPlan.ItineraryJson,
58                  Places = System.Text.Json.JsonSerializer.
    Deserialize<List<FetchedPlaceDto>>(dbCachedPlan.PlacesJson) ?? [],
59                  TotalPlacesFound = System.Text.Json.JsonSerializer.
    Deserialize<List<FetchedPlaceDto>>(dbCachedPlan.PlacesJson)?.Count
    ?? 0
60              };
61
62              // Repopulate memory cache
63              _cache.Set(cacheKey, responseFromDb, TimeSpan.FromHours
    (24));
64              return responseFromDb;
65          }
66
67          // 2. Center around specific landmark if provided
68          if (request.CenterPlaceId.HasValue && request.CenterPlaceId
     > 0)
69          {
70              var centerPlace = await _unitOfWork.Repository<Place>()
    .GetByIdAsync(request.CenterPlaceId.Value);
71              if (centerPlace != null)
72              {
73                  request.Latitude = (double)centerPlace.Latitude;
74                  request.Longitude = (double)centerPlace.Longitude;
75              }
76          }
77
78          // 3. Smart Fetching Strategy: Local DB first     strict
    proximity to avoid cross-city results
79          // Cap at 30km regardless of requested radius to ensure
    city-accurate results
```

```
80          var strictRadiusKm = Math.Min(request.MaxRadiusKm, 30.0);
81          var allDbPlaces = await _unitOfWork.Repository<Place>().
    GetAllAsync();
82          var localPlaces = allDbPlaces
83              .Select(p => new FetchedPlaceDto
84              {
85                  ExternalId = p.FoursquareId,
86                  Name = p.Name,
87                  Latitude = (double)p.Latitude,
88                  Longitude = (double)p.Longitude,
89                  Address = p.Address ?? "",
90                  ImageUrl = p.MainImageURL ?? "",
91                  Rating = (double)p.Rating,
92                  PriceLevel = p.PriceLevel.ToString(),
93                  Types = new List<string> { p.Source ?? "db" },
94                  DistanceKm = Math.Round(HaversineKm(request.
    Latitude, request.Longitude, (double)p.Latitude, (double)p.Longitude
    ), 2)
95              })
96              // Only include places within strict radius AND that
    have valid coordinates (non-zero)
97              .Where(p => p.DistanceKm <= strictRadiusKm && (p.
    Latitude != 0 || p.Longitude != 0))
98              .OrderBy(p => p.DistanceKm)
99              .ToList();
100
101         _logger.LogInformation("[TripPlanner] Found {Count} local
    DB places within {Radius}km of {Location}",
102             localPlaces.Count, strictRadiusKm, request.Location);
103
104         List<FetchedPlaceDto> finalPlaces = new List<
    FetchedPlaceDto>(localPlaces);
105
106         if (finalPlaces.Count < 15)
107         {
108             // Trigger Foursquare API to fill gaps
109             var fsPlaces = await _placesDataService.
    FetchNearbyPlacesAsync(
110                 request.Latitude, request.Longitude,
```

```
111                     request.MinRadiusKm, request.MaxRadiusKm);
112
113                 // Merge Foursquare results, avoiding duplicates with
    DB places
114                 foreach (var fsp in fsPlaces)
115                 {
116                     if (!finalPlaces.Any(lp => lp.Name.Equals(fsp.Name,
     StringComparison.OrdinalIgnoreCase)))
117                     {
118                         finalPlaces.Add(fsp);
119
120                         // Async persist to DB with RICH ATTRIBUTES
121                         var newPlace = new Place
122                         {
123                             FoursquareId = fsp.ExternalId,
124                             Name = fsp.Name,
125                             Address = fsp.Address,
126                             Latitude = (decimal)fsp.Latitude,
127                             Longitude = (decimal)fsp.Longitude,
128                             Rating = (decimal)(fsp.Rating ?? 0),
129                             PriceLevel = int.TryParse(fsp.PriceLevel,
    out int pl) ? pl : 0,
130                             MainImageURL = fsp.ImageUrl,
131                             Source = "foursquare",
132                             LastEnrichedAt = DateTime.UtcNow
133                         };
134                         await _unitOfWork.Repository<Place>().AddAsync(
    newPlace);
135                     }
136                 }
137                 await _unitOfWork.CommitAsync();
138             }
139
140             // Sequential image fetching removed to massively improve
    API response time.
141             // Images are now fetched concurrently only for places the
    AI actually selected.
142
143             if (finalPlaces.Count == 0)
```

```
144             {
145                 return new TripPlanResponseDto
146                 {
147                     TotalPlacesFound = 0,
148                     Places = [],
149                     TripPlan = "No places found in the specified area.
    Try enlarging the radius."
150                 };
151             }
152 
153             // Build a rich place list grouping by type for better AI
    context
154             var hotels = finalPlaces.Where(p => (p.Categories ?? p.
    Types).Any(t =>
155                 t.Contains("hotel", StringComparison.OrdinalIgnoreCase)
     ||
156                 t.Contains("hostel", StringComparison.OrdinalIgnoreCase
    ) ||
157                 t.Contains("accommodation", StringComparison.
    OrdinalIgnoreCase) ||
158                 t.Contains("resort", StringComparison.OrdinalIgnoreCase
    ))).ToList();
159 
160             var restaurants = finalPlaces.Where(p => (p.Categories ?? p
    .Types).Any(t =>
161                 t.Contains("restaurant", StringComparison.
    OrdinalIgnoreCase) ||
162                 t.Contains("food", StringComparison.OrdinalIgnoreCase)
    ||
163                 t.Contains("dining", StringComparison.OrdinalIgnoreCase
    ) ||
164                 t.Contains("bistro", StringComparison.OrdinalIgnoreCase
    ))).ToList();
165 
166             var cafes = finalPlaces.Where(p => (p.Categories ?? p.Types
    ).Any(t =>
167                 t.Contains("cafe", StringComparison.OrdinalIgnoreCase)
    ||
```

```
168                 t.Contains("caf ", StringComparison.OrdinalIgnoreCase)
     ||
169                 t.Contains("coffee", StringComparison.OrdinalIgnoreCase
    ) ||
170                 t.Contains("bakery", StringComparison.OrdinalIgnoreCase
    ))).ToList();
171
172             var attractions = finalPlaces.Except(hotels).Except(
    restaurants).Except(cafes).ToList();
173
174             // Combine: put hotels first, then top attractions, then
    dining
175             var orderedPlaces = hotels.Take(3)
176                 .Concat(attractions.OrderByDescending(p => p.Rating).
    Take(20))
177                 .Concat(restaurants.OrderByDescending(p => p.Rating).
    Take(8))
178                 .Concat(cafes.OrderByDescending(p => p.Rating).Take(4))
179                 .DistinctBy(p => p.Name)
180                 .Take(35)
181                 .ToList();
182
183             if (orderedPlaces.Count == 0) orderedPlaces = finalPlaces.
    Take(35).ToList();
184
185             var placesListText = string.Join("\n", orderedPlaces.Select
    ((p, idx) =>
186             {
187                 var typeLabel = hotels.Any(h => h.Name == p.Name) ? "[
    HOTEL]" :
188                                 restaurants.Any(r => r.Name == p.Name)
    ? "[RESTAURANT]" :
189                                 cafes.Any(c => c.Name == p.Name) ? "[
    CAF ]" : "[ATTRACTION]";
190                 var cats = string.Join(", ", (p.Categories ?? p.Types).
    Take(2));
191                 return $"{idx + 1}. {typeLabel} {p.Name} | Rating: {p.
    Rating:F1} | {cats} | {p.Address}";
192             }));
```

```
193
194             var sysPrompt = @"You are Kemora, an expert Egyptian travel
    planner building premium itineraries.
195 You MUST output ONLY valid JSON, exactly matching this structure:
196 {
197   ""itinerary"": [
198     {
199       ""day"": 1,
200       ""theme"": ""Theme for the day e.g. Historic Cairo & Culture"",
201       ""activities"": [
202         { ""time"": ""08:00"", ""time_slot"": ""Morning"", ""place"": "
    "Exact Name From List"", ""description"": ""2-3 sentence vivid
    description of what to do there and why it is special."" },
203         { ""time"": ""10:30"", ""time_slot"": ""Morning"", ""place"": "
    "Exact Name From List"", ""description"": ""..."" },
204         { ""time"": ""13:00"", ""time_slot"": ""Afternoon"", ""place"":
     ""Restaurant Name"", ""description"": ""Lunch stop description.""
    },
205         { ""time"": ""15:00"", ""time_slot"": ""Afternoon"", ""place"":
     ""Exact Name From List"", ""description"": ""..."" },
206         { ""time"": ""17:30"", ""time_slot"": ""Afternoon"", ""place"":
     ""Cafe Name"", ""description"": ""Afternoon coffee break."" },
207         { ""time"": ""19:30"", ""time_slot"": ""Evening"", ""place"": "
    "Restaurant Name"", ""description"": ""Dinner description."" }
208       ]
209     }
210   ]
211 }
212
213 RULES:
214 - Use EXACT place names from the provided list only
215 - Assign realistic times: Morning (07:00-12:00), Afternoon
    (12:00-18:00), Evening (18:00-23:00)
216 - time_slot MUST be one of: Morning, Afternoon, Evening
217 - Each day MUST include: 1 hotel check-in (day 1 only), 1-2 restaurants
     for meals, 3-5 sightseeing attractions, 1 cafe break
218 - Write rich engaging 2-3 sentence descriptions for each activity
219 - Spread activities logically across the full day";
220
```

```
221             var userPrompt = $@"Generate a {request.DurationDays}-day
    itinerary for {request.Location}, Egypt.
222 Budget: {request.Budget}. Interests: {request.TourismTypes}. User
    preferences: {prefs}.
223 Ensure each day has Morning + Afternoon + Evening activities with
    realistic times.
224 Select ONLY from this curated list:
225
226 {placesListText}";
227
228
229             var tripPlanContent = await _aiService.
    GenerateCompletionAsync(sysPrompt, userPrompt, jsonMode: true);
230
231             if (string.IsNullOrEmpty(tripPlanContent) ||
    tripPlanContent.StartsWith("AI Error:"))
232             {
233                 _logger.LogError("AI Trip Planning failed: {Error}",
    tripPlanContent);
234                 return new TripPlanResponseDto
235                 {
236                     TotalPlacesFound = finalPlaces.Count,
237                     Places = finalPlaces,
238                     TripPlan = "Sorry, our AI travel planner is
    currently busy or rate-limited. Please try again in a few moments."
239                 };
240             }
241
242             // Clean response content (OpenRouter models usually return
     clean JSON, but strip markdown blocks just in case)
243             tripPlanContent = tripPlanContent.Trim();
244             if (tripPlanContent.StartsWith("```"))
245             {
246                 var startIdx = tripPlanContent.IndexOf('\n') + 1;
247                 var endIdx = tripPlanContent.LastIndexOf("```");
248                 if (endIdx > startIdx)
249                 {
250                     tripPlanContent = tripPlanContent.Substring(
    startIdx, endIdx - startIdx).Trim();
```

```
                }
            }

            // Re-inject image URLs into the itinerary from the local loaded places
            try
            {
                var jObject = System.Text.Json.Nodes.JsonObject.Parse(tripPlanContent);
                var itinerary = jObject?["itinerary"]?.AsArray();
                if (itinerary != null)
                {
                    foreach (var day in itinerary)
                    {
                        var activities = day?["activities"]?.AsArray();
                        if (activities != null)
                        {
                            foreach (var act in activities)
                            {
                                var placeName = act?["place"]?.ToString();
                                if (!string.IsNullOrEmpty(placeName))
                                {
                                    var match = finalPlaces.FirstOrDefault(p =>
                                        p.Name.Equals(placeName, StringComparison.OrdinalIgnoreCase));
                                    if (match != null && !string.IsNullOrEmpty(match.ImageUrl) && !match.ImageUrl.Contains("placeholder"))
                                    {
                                        act["image_url"] = match.ImageUrl;
                                    }
                                }
                            }
                        }
                    }
                }
```

```
282                 tripPlanContent = jObject?.ToJsonString() ??
    tripPlanContent;
283             }
284             catch { /* Ignore image injection errors     the clean JSON
     is still valid */ }
285 
286             var response = new TripPlanResponseDto
287             {
288                 TotalPlacesFound = finalPlaces.Count,
289                 Places = finalPlaces,
290                 TripPlan = tripPlanContent
291             };
292 
293             // 5. Save the generated alternative to DB (Level 2)
294             var newDbCache = new PrecomputedTripPlan
295             {
296                 CacheKey = cacheKey,
297                 ItineraryJson = tripPlanContent,
298                 PlacesJson = System.Text.Json.JsonSerializer.Serialize(
    finalPlaces),
299                 CreatedAt = DateTime.UtcNow
300             };
301             await _unitOfWork.Repository<PrecomputedTripPlan>().
    AddAsync(newDbCache);
302             await _unitOfWork.CommitAsync();
303 
304             // 6. Save the generated alternative in Redis/Memory Cache
    (Level 1)
305             _cache.Set(cacheKey, response, TimeSpan.FromHours(24));
306 
307             return response;
308         }
309 
310         public async Task<string> SwapPlaceAsync(string
    currentPlaceName, string preferences)
311         {
312             var sysPrompt = @"You are a professional Egyptian travel
    assistant.
```

```
313 Your task is to swap a specific place in a trip itinerary with a better
        alternative.
314 You MUST return valid JSON only, exactly matching this structure:
315 {
316   ""newActivity"": {
317     ""time"": ""HH:mm"",
318     ""place"": ""New Place Name"",
319     ""description"": ""Detailed description of why this is a good
       alternative.""
320   }
321 }";
322             var userPrompt = $@"The current place/activity is: '{
       currentPlaceName}'.
323 User preferences/reason for swapping: '{preferences ?? "looking for
       something better"}'.
324 Please suggest a HIGH-QUALITY alternative in the same city/area that
       matches these preferences.
325 Ensure the JSON is clean and valid.";
326
327             var result = await _aiService.GenerateCompletionAsync(
       sysPrompt, userPrompt, jsonMode: true);
328
329             // Basic fallback if AI fails or returns non-JSON
330             if (string.IsNullOrEmpty(result) || result.StartsWith("AI
       Error:"))
331             {
332                 return "{\"newActivity\": {\"time\": \"09:00\", \"place
       \": \"Alternative Suggestion\", \"description\": \"Sorry, I could
       not generate a specific alternative at this moment. Please try again
        or select another place manually.\"}}";
333             }
334
335             return result;
336         }
337
338         private static double HaversineKm(double lat1, double lng1,
       double lat2, double lng2)
339         {
340             const double R = 6371;
```

```
            var dLat = ToRad(lat2 - lat1);
            var dLng = ToRad(lng2 - lng1);
            var a = Math.Sin(dLat / 2) * Math.Sin(dLat / 2)
                  + Math.Cos(ToRad(lat1)) * Math.Cos(ToRad(lat2))
                  * Math.Sin(dLng / 2) * Math.Sin(dLng / 2);
            return R * 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
        }

        private static double ToRad(double deg) => deg * Math.PI / 180;
    }
}
```

B.4 Program.cs (Backend Configuration)

The central entry point of the Kemora backend API. This setup demonstrates the extensive middleware pipeline employed, including rate limiting, JWT authentication, identity management, Serilog-based structural logging, and the dependency injection bindings required to cleanly orchestrate the Domain and Application layers.

```
using Kemora.Domain.Entities;
using Kemora.Domain.Interfaces;
using Kemora.Infrastructure.Data;
using Kemora.Infrastructure.Services;
using Microsoft.AspNetCore.Mvc;
using System.Linq;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi;
using Microsoft.OpenApi.Models;
using System.Text;
using System.Net.Http.Headers;
using System.Threading.RateLimiting;
using Serilog;
using dotenv.net;

DotEnv.Load();
```

```
20
21 Log.Logger = new LoggerConfiguration()
22     .MinimumLevel.Information()
23     .MinimumLevel.Override("Microsoft", Serilog.Events.LogEventLevel.
   Warning)
24     .MinimumLevel.Override("Microsoft.EntityFrameworkCore", Serilog.
   Events.LogEventLevel.Warning)
25     .WriteTo.Console()
26     .WriteTo.File("logs/kemora-.log", rollingInterval: RollingInterval.
   Day, retainedFileCountLimit: 14)
27     .Enrich.FromLogContext()
28     .CreateLogger();
29
30 try
31 {
32
33 var builder = WebApplication.CreateBuilder(args);
34 builder.Host.UseSerilog();
35
36 // 1. Database Configuration
37 builder.Services.AddDbContext<ApplicationDbContext>(options =>
38     options.UseSqlServer(builder.Configuration.GetConnectionString("
   DefaultConnection")));
39
40 builder.Services.Configure<Kemora.Application.DTOs.EmailSettings>(
   builder.Configuration.GetSection("EmailSettings"));
41
42 // 2. Identity (User Management) Config
43 builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
44 {
45     // Production password settings
46     options.User.RequireUniqueEmail = true;
47     options.Password.RequireDigit = true;
48     options.Password.RequiredLength = 8;
49     options.Password.RequireNonAlphanumeric = true;
50     options.Password.RequireUppercase = true;
51     options.Password.RequireLowercase = true;
52 })
53 .AddEntityFrameworkStores<ApplicationDbContext>()
```

```
54 .AddDefaultTokenProviders();
55
56 // 3. JWT Authentication Config
57 var tokenKey = builder.Configuration["TokenKey"] ?? "
       super_secret_key_must_be_long_enough_for_security";
58 var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(tokenKey));
59
60 builder.Services.AddAuthentication(options =>
61 {
62     options.DefaultAuthenticateScheme = JwtBearerDefaults.
       AuthenticationScheme;
63     options.DefaultChallengeScheme = JwtBearerDefaults.
       AuthenticationScheme;
64 })
65 .AddJwtBearer(options =>
66 {
67     options.TokenValidationParameters = new TokenValidationParameters
68     {
69         ValidateIssuerSigningKey = true,
70         IssuerSigningKey = key,
71         ValidateIssuer = false,   // Set to true in production
72         ValidateAudience = false  // Set to true in production
73     };
74 });
75
76 // 4. Register Services (Dependency Injection)
77
78 // Domain / Infrastructure Repositories
79 builder.Services.AddScoped(typeof(Kemora.Domain.Interfaces.IRepository
       <>), typeof(Kemora.Infrastructure.Repositories.Repository<>));
80 builder.Services.AddScoped<Kemora.Domain.Interfaces.IPostRepository,
       Kemora.Infrastructure.Repositories.PostRepository>();
81 builder.Services.AddScoped<Kemora.Domain.Interfaces.ICommentRepository,
       Kemora.Infrastructure.Repositories.CommentRepository>();
82 builder.Services.AddMemoryCache();
83 builder.Services.AddScoped<Kemora.Application.Interfaces.ICacheService,
       Kemora.Infrastructure.Services.MemoryCacheService>();
84 builder.Services.AddScoped<Kemora.Domain.Interfaces.IReactionRepository
       , Kemora.Infrastructure.Repositories.ReactionRepository>();
```

```
85 builder.Services.AddScoped<Kemora.Domain.Interfaces.IUnitOfWork, Kemora
       .Infrastructure.Repositories.UnitOfWork>();
86 builder.Services.AddScoped<Kemora.Domain.Interfaces.ITripRepository,
       Kemora.Infrastructure.Repositories.TripRepository>();
87 builder.Services.AddScoped<Kemora.Domain.Interfaces.IPlaceRepository,
       Kemora.Infrastructure.Repositories.PlaceRepository>();
88 builder.Services.AddScoped<Kemora.Domain.Interfaces.IReviewRepository,
       Kemora.Infrastructure.Repositories.ReviewRepository>();
89 builder.Services.AddScoped<Kemora.Domain.Interfaces.IPhotoRepository,
       Kemora.Infrastructure.Repositories.PhotoRepository>();
90 builder.Services.AddScoped<Kemora.Domain.Interfaces.IEventRepository,
       Kemora.Infrastructure.Repositories.EventRepository>();
91 builder.Services.AddScoped<Kemora.Domain.Interfaces.IFavoriteRepository
       , Kemora.Infrastructure.Repositories.FavoriteRepository>();
92 builder.Services.AddScoped<Kemora.Domain.Interfaces.IBadgeRepository,
       Kemora.Infrastructure.Repositories.BadgeRepository>();
93 builder.Services.AddScoped<Kemora.Domain.Interfaces.
       INotificationRepository, Kemora.Infrastructure.Repositories.
       NotificationRepository>();
94 builder.Services.AddScoped<Kemora.Domain.Interfaces.IUserRepository,
       Kemora.Infrastructure.Repositories.UserRepository>();
95
96 // Application Services
97 builder.Services.AddScoped<Kemora.Domain.Interfaces.ITokenService,
       Kemora.Infrastructure.Services.TokenService>();
98
99 builder.Services.AddScoped<Kemora.Domain.Interfaces.IPlacesDataService,
        Kemora.Infrastructure.Services.GooglePlacesService>();
100 builder.Services.AddSingleton<Kemora.Domain.Interfaces.IAiService,
       Kemora.Infrastructure.Services.OpenRouterAiService>();
101 builder.Services.AddScoped<Kemora.Application.Interfaces.IAuthService,
       Kemora.Infrastructure.Services.AuthService>();
102 builder.Services.AddScoped<Kemora.Application.Interfaces.IBadgeService,
        Kemora.Application.Services.BadgeService>();
103 builder.Services.AddScoped<Kemora.Application.Interfaces.IChatService,
       Kemora.Application.Services.ChatService>();
104 // builder.Services.AddScoped<Kemora.Domain.Interfaces.
       IWikipediaService, Kemora.Infrastructure.Services.WikipediaService
       >();
```

```
105 builder.Services.AddScoped<Kemora.Application.Interfaces.IEmailService,
        Kemora.Infrastructure.Services.SmtpEmailService>();
106 builder.Services.AddScoped<Kemora.Application.Interfaces.IImageService,
        Kemora.Infrastructure.Services.CloudinaryImageService>();
107 builder.Services.AddScoped<Kemora.Application.Interfaces.
    ICommentService, Kemora.Application.Services.CommentService>();
108 builder.Services.AddScoped<Kemora.Application.Interfaces.IEventService,
        Kemora.Application.Services.EventService>();
109 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IFavoriteService, Kemora.Application.Services.FavoriteService>();
110 builder.Services.AddScoped<Kemora.Application.Interfaces.
    INotificationService, Kemora.Application.Services.
    NotificationService>();
111 builder.Services.AddScoped<Kemora.Application.Interfaces.IPhotoService,
        Kemora.Application.Services.PhotoService>();
112 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IPlaceManagementService, Kemora.Application.Services.
    PlaceManagementService>();
113 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IPlacePublicService, Kemora.Application.Services.PlacePublicService
    >();
114 builder.Services.AddScoped<Kemora.Application.Interfaces.IPostService,
    Kemora.Application.Services.PostService>();
115 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IProfileService, Kemora.Infrastructure.Services.ProfileService>();
116 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IReactionService, Kemora.Application.Services.ReactionService>();
117 builder.Services.AddScoped<Kemora.Application.Interfaces.IReviewService
    , Kemora.Application.Services.ReviewService>();
118 builder.Services.AddScoped<Kemora.Application.Interfaces.ITripService,
    Kemora.Application.Services.TripService>();
119 builder.Services.AddScoped<Kemora.Application.Interfaces.
    ITripPlannerService, Kemora.Application.Services.TripPlannerService
    >();
120 builder.Services.AddScoped<Kemora.Application.Interfaces.
    IUserManagementService, Kemora.Infrastructure.Services.
    UserManagementService>();
121 builder.Services.AddScoped<Kemora.Domain.Interfaces.IStoryRepository,
    Kemora.Infrastructure.Repositories.StoryRepository>();
```

```
builder.Services.AddScoped<Kemora.Application.Interfaces.IStoryService,
    Kemora.Application.Services.StoryService>();
builder.Services.AddScoped<Kemora.Application.Interfaces.
    IBadgeAwardService, Kemora.Infrastructure.Services.BadgeAwardService
    >();

// SignalR
builder.Services.AddSignalR();
builder.Services.AddScoped<Kemora.Application.Interfaces.
    INotificationPusher, Kemora.Api.Services.SignalRNotificationPusher
    >();

// Rate Limiting
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = 429;
    options.AddPolicy("fixed", context =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: context.Connection.RemoteIpAddress?.ToString
    () ?? "unknown",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 60,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0
            }));
    options.AddPolicy("auth", context =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: context.Connection.RemoteIpAddress?.ToString
    () ?? "unknown",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 10,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0
            }));
});

// Health Checks
```

```
154 builder.Services.AddHealthChecks()
155     .AddSqlServer(builder.Configuration.GetConnectionString("
    DefaultConnection")!);
156
157 // Response Caching
158 builder.Services.AddResponseCaching();
159
160 // AutoMapper
161 builder.Services.AddAutoMapper(cfg => {
162     cfg.AddProfile<Kemora.Application.Mapping.MappingProfile>();
163 });
164
165 builder.Services.AddHttpClient("GooglePlaces", client =>
166 {
167     client.Timeout = TimeSpan.FromSeconds(30);
168 });
169
170
171
172 builder.Services.AddHttpClient("OpenRouter", client =>
173 {
174     client.BaseAddress = new Uri("https://openrouter.ai/api/v1/");
175     client.DefaultRequestHeaders.Accept.Add(new
    MediaTypeWithQualityHeaderValue("application/json"));
176     client.DefaultRequestHeaders.Add("X-OpenRouter-Title", "Kemora
    Travel Planner");
177     client.Timeout = TimeSpan.FromMinutes(3);
178 });
179
180 builder.Services.AddControllers()
181     .ConfigureApiBehaviorOptions(options =>
182     {
183         options.InvalidModelStateResponseFactory = context =>
184         {
185             var errors = context.ModelState
186                 .Where(e => e.Value!.Errors.Count > 0)
187                 .ToDictionary(
188                     kvp => kvp.Key,
```

```
                    kvp => kvp.Value!.Errors.Select(e => e.ErrorMessage
    ).ToArray()
                );

            return new BadRequestObjectResult(new
            {
                Message = "One or more validation errors occurred.",
                Errors = errors
            });
        };
    }); // We use Controllers, not Minimal APIs

builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new Asp.Versioning.ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
}).AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});

// 5. Swagger / OpenAPI Config (With Auth Support)
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "Kemora Tourism API",
        Version = "v1",
        Description = "RESTful API for the Kemora tourism platform
    explore Egyptian destinations, plan trips, engage with the community
    , and earn gamification badges.",
        Contact = new OpenApiContact
        {
            Name = "Kemora Team",
            Email = "support@kemora.app"
        },
```

```
225         License = new OpenApiLicense
226         {
227             Name = "MIT License"
228         }
229     });
230 
231     // Include XML comments from API project
232     var xmlFilename = $"{System.Reflection.Assembly.
    GetExecutingAssembly().GetName().Name}.xml";
233     var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFilename);
234     if (File.Exists(xmlPath)) c.IncludeXmlComments(xmlPath);
235 
236     // Enable the "Authorize" button in Swagger UI
237     c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
238     {
239         Description = "JWT Authorization header using the Bearer scheme
    . \r\n\r\n Enter 'Bearer' [space] and then your token in the text
    input below.\r\n\r\nExample: 'Bearer 12345abcdef'",
240         Name = "Authorization",
241         In = ParameterLocation.Header,
242         Type = SecuritySchemeType.ApiKey,
243         Scheme = "Bearer"
244     });
245 
246     c.AddSecurityRequirement(new OpenApiSecurityRequirement()
247     {
248         {
249             new OpenApiSecurityScheme
250             {
251                 Reference = new OpenApiReference
252                 {
253                     Type = ReferenceType.SecurityScheme,
254                     Id = "Bearer"
255                 },
256                 Scheme = "oauth2",
257                 Name = "Bearer",
258                 In = ParameterLocation.Header,
259             },
260             new List<string>()
```

```
        }
    });
});

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.SetIsOriginAllowed(_ => true) // Allows Flutter Web's random localhost ports
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

var app = builder.Build();

app.UseMiddleware<Kemora.Api.Middlewares.ExceptionHandlingMiddleware>();
app.UseSerilogRequestLogging();

// 6. Middleware Pipeline

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

// Skip HTTPS redirection in development so Flutter Web (Chrome) can call HTTP port 5299
if (!app.Environment.IsDevelopment())
{
    app.UseHttpsRedirection();
}

app.UseCors("AllowFrontend");
```

```
297
298 // Serve uploaded images from wwwroot/uploads as static files
299 app.UseStaticFiles();
300
301 // IMPORTANT: Authentication must come BEFORE Authorization
302 app.UseAuthentication();
303 app.UseAuthorization();
304 app.UseResponseCaching();
305
306 app.MapControllers();
307 app.MapHub<Kemora.Api.Hubs.NotificationHub>("/hubs/notifications");
308 app.MapHealthChecks("/health");
309 app.UseRateLimiter();
310
311 using (var scope = app.Services.CreateScope())
312 {
313     await RoleSeeder.SeedRolesAsync(scope.ServiceProvider);
314
315     if (app.Environment.IsDevelopment())
316     {
317         var context = scope.ServiceProvider.GetRequiredService<
    ApplicationDbContext>();
318
319         try
320         {
321             Log.Information("DATABASE STARTUP: Ensuring base data is
    seeded...");
322             await DataSeeder.SeedAsync(scope.ServiceProvider);
323
324             var placeCount = await context.Places.CountAsync();
325             Log.Information("DATABASE STARTUP: Ready. Current Place
    count: {Count}", placeCount);
326         }
327         catch (Exception ex)
328         {
329             Log.Error(ex, "DATABASE STARTUP: Error during seeding check
    ");
330         }
331     }
```

```
}

app.Run();

}
catch (Exception ex)
{
    Log.Fatal(ex, "Application terminated unexpectedly");
}
finally
{
    Log.CloseAndFlush();
}

public partial class Program { }
```

B.5 TripViewModel.dart (Flutter State Management)

The primary state management ViewModel in the Flutter frontend for orchestrating trip planning. It handles fetching user trips, deleting or renaming plans, and deeply interacts with the AI itinerary generation use-cases. Through Flutter's `ChangeNotifier` pattern, it achieves reactive UI updates when items are reordered, deleted, or their visited status is toggled.

```
import 'package:flutter/material.dart';

import '../../domain/entities/trip.dart';
import '../../domain/usecases/trip_usecases.dart';
import '../../domain/usecases/generate_ai_itinerary_usecase.dart';
import '../../domain/usecases/swap_place_usecase.dart';
import '../../domain/usecases/save_ai_plan_usecase.dart';
import '../../domain/usecases/update_place_visited_status_usecase.dart
    ';
import '../../domain/entities/ai_itinerary.dart';
import '../../domain/entities/trip_plan_request.dart';

enum TripState { initial, loading, loaded, error }

```

```
14 class TripViewModel extends ChangeNotifier {
15   final GetUserTripsUseCase getUserTripsUseCase;
16   final CreateTripPlanUseCase createTripPlanUseCase;
17   final GenerateAiItineraryUseCase generateAiItineraryUseCase;
18   final SwapPlaceUseCase swapPlaceUseCase;
19   final SaveAiPlanUseCase saveAiPlanUseCase;
20   final UpdatePlaceVisitedStatusUseCase updatePlaceVisitedStatusUseCase
      ;
21   final GetTripDetailsUseCase getTripDetailsUseCase;
22   final DeleteTripUseCase deleteTripUseCase;
23   final RenameTripUseCase renameTripUseCase;
24
25   TripViewModel({
26     required this.getUserTripsUseCase,
27     required this.createTripPlanUseCase,
28     required this.generateAiItineraryUseCase,
29     required this.swapPlaceUseCase,
30     required this.saveAiPlanUseCase,
31     required this.updatePlaceVisitedStatusUseCase,
32     required this.getTripDetailsUseCase,
33     required this.deleteTripUseCase,
34     required this.renameTripUseCase,
35   });
36
37   TripState _state = TripState.initial;
38   TripState get state => _state;
39
40   List<Trip> _trips = [];
41   List<Trip> get trips => _trips;
42
43   AIItinerary? _currentPlan;
44   AIItinerary? get currentPlan => _currentPlan;
45
46   String? _errorMessage;
47   String? get errorMessage => _errorMessage;
48
49   void setCurrentPlan(AIItinerary? plan) {
50     _currentPlan = plan;
51     notifyListeners();
```

```
52  }
53
54  Future<void> loadTrips() async {
55    _state = TripState.loading;
56    _errorMessage = null;
57    notifyListeners();
58
59    print('=== LOAD_TRIPS: Calling getUserTripsUseCase ===');
60    final result = await getUserTripsUseCase();
61
62    result.fold(
63      (failure) {
64        print('=== LOAD_TRIPS ERROR: ${failure.message} ===');
65        _state = TripState.error;
66        _errorMessage = failure.message;
67        notifyListeners();
68      },
69      (tripsList) {
70        print('=== LOAD_TRIPS SUCCESS: Got ${tripsList.length} trips
     ===');
71        for (final t in tripsList) {
72          print('  Trip: id="${t.id}" title="${t.title}"');
73        }
74        _trips = tripsList;
75        _state = TripState.loaded;
76        notifyListeners();
77      },
78    );
79  }
80
81  Future<Trip?> loadTripDetails(String id) async {
82    final result = await getTripDetailsUseCase(id);
83    return result.fold(
84      (failure) {
85        _errorMessage = failure.message;
86        notifyListeners();
87        return null;
88      },
89      (trip) {
```

```
90        final index = _trips.indexWhere((t) => t.id == id);
91        if (index != -1) {
92          _trips[index] = trip;
93          notifyListeners();
94        }
95        return trip;
96      },
97    );
98  }
99
100 Future<bool> deleteTrip(String id) async {
101   print('=== VM.deleteTrip: id="$id" ===');
102   final result = await deleteTripUseCase(id);
103   return result.fold(
104     (failure) {
105       print('=== VM.deleteTrip ERROR: ${failure.message} ===');
106       _errorMessage = failure.message;
107       notifyListeners();
108       return false;
109     },
110     (_) {
111       print('=== VM.deleteTrip SUCCESS ===');
112       _trips.removeWhere((t) => t.id == id);
113       notifyListeners();
114       return true;
115     },
116   );
117 }
118
119 Future<bool> renameTrip(String id, String newName) async {
120   print('=== VM.renameTrip: id="$id" newName="$newName" ===');
121   final result = await renameTripUseCase(id, newName);
122   return result.fold(
123     (failure) {
124       print('=== VM.renameTrip ERROR: ${failure.message} ===');
125       _errorMessage = failure.message;
126       notifyListeners();
127       return false;
128     },
```

```
129       (_) {
130         print('=== VM.renameTrip SUCCESS ===');
131         final index = _trips.indexWhere((t) => t.id == id);
132         if (index != -1) {
133           final oldTrip = _trips[index];
134           _trips[index] = Trip(
135             id: oldTrip.id,
136             title: newName,
137             startDate: oldTrip.startDate,
138             endDate: oldTrip.endDate,
139             plannedPlaces: oldTrip.plannedPlaces,
140             savedItinerary: oldTrip.savedItinerary,
141             location: oldTrip.location,
142           );
143           notifyListeners();
144         }
145         return true;
146       },
147     );
148   }
149
150   Future<void> generateAiItinerary(TripPlanRequest request) async {
151     _state = TripState.loading;
152     _errorMessage = null;
153     _currentPlan = null;
154     notifyListeners();
155
156     final result = await generateAiItineraryUseCase(request);
157
158     result.fold(
159       (failure) {
160         _state = TripState.error;
161         _errorMessage = failure.message;
162         notifyListeners();
163       },
164       (itinerary) {
165         final locationTitle = (request.location?.isNotEmpty == true) ?
    'Trip to ${request.location}' : 'AI Trip Plan';
166         _currentPlan = AIItinerary(
```

```
        title: itinerary.title == 'AI Trip Plan' || itinerary.title.isEmpty ? locationTitle : itinerary.title,
        duration: itinerary.duration,
        days: itinerary.days,
      );
      _state = TripState.loaded;
      notifyListeners();
    },
  );
}

Future<void> swapPlace(String currentPlaceName, String preferences) async {
  if (_currentPlan == null) return;

  final result = await swapPlaceUseCase(currentPlaceName, preferences);

  result.fold(
    (failure) {
      _errorMessage = failure.message;
      notifyListeners();
    },
    (newItem) {
      final updatedDays = _currentPlan!.days.map((day) {
        final updatedActivities = day.activities.map((activity) {
          if (activity.name == currentPlaceName) {
            return newItem;
          }
          return activity;
        }).toList();
        return TripDay(dayNumber: day.dayNumber, activities: updatedActivities);
      }).toList();

      _currentPlan = AIItinerary(
        title: _currentPlan!.title,
        duration: _currentPlan!.duration,
        days: updatedDays,
```

```
      );
      notifyListeners();
    },
  );
}

void removePlace(int dayIndex, int activityIndex) {
  if (_currentPlan == null) return;

  final updatedDays = List<TripDay>.from(_currentPlan!.days);
  if (dayIndex >= 0 && dayIndex < updatedDays.length) {
    final updatedActivities = List<ItineraryItem>.from(updatedDays[
 dayIndex].activities);
    if (activityIndex >= 0 && activityIndex < updatedActivities.
 length) {
      updatedActivities.removeAt(activityIndex);
      updatedDays[dayIndex] = TripDay(
        dayNumber: updatedDays[dayIndex].dayNumber,
        activities: updatedActivities,
        dailySummary: updatedDays[dayIndex].dailySummary,
        transportTips: updatedDays[dayIndex].transportTips,
      );

      _currentPlan = AIItinerary(
        title: _currentPlan!.title,
        duration: _currentPlan!.duration,
        days: updatedDays,
      );
      notifyListeners();
    }
  }
}

void reorderPlace(int dayIndex, int oldIndex, int newIndex) {
  if (_currentPlan == null) return;

  final updatedDays = List<TripDay>.from(_currentPlan!.days);
  if (dayIndex >= 0 && dayIndex < updatedDays.length) {
```

```
238       final updatedActivities = List<ItineraryItem>.from(updatedDays[
    dayIndex].activities);
239       if (oldIndex >= 0 && oldIndex < updatedActivities.length) {
240         if (newIndex > oldIndex) {
241           newIndex -= 1;
242         }
243         final item = updatedActivities.removeAt(oldIndex);
244         updatedActivities.insert(newIndex, item);
245 
246         updatedDays[dayIndex] = TripDay(
247           dayNumber: updatedDays[dayIndex].dayNumber,
248           activities: updatedActivities,
249           dailySummary: updatedDays[dayIndex].dailySummary,
250           transportTips: updatedDays[dayIndex].transportTips,
251         );
252 
253         _currentPlan = AIItinerary(
254           title: _currentPlan!.title,
255           duration: _currentPlan!.duration,
256           days: updatedDays,
257         );
258         notifyListeners();
259       }
260     }
261   }
262 
263   Future<bool> savePlan(DateTime startDate, DateTime endDate) async {
264     if (_currentPlan == null) return false;
265 
266     _state = TripState.loading;
267     _errorMessage = null;
268     notifyListeners();
269 
270     print('=== VM.savePlan: title="${_currentPlan!.title}" ===');
271     final result = await saveAiPlanUseCase(_currentPlan!, startDate,
    endDate);
272 
273     return result.fold(
274       (failure) {
```

```
275         print('=== VM.savePlan ERROR: ${failure.message} ===');
276         _state = TripState.error;
277         _errorMessage = failure.message;
278         notifyListeners();
279         return false;
280       },
281       (trip) {
282         print('=== VM.savePlan SUCCESS: tripId="${trip.id}" title="${
    trip.title}" ===');
283         _trips.add(trip);
284         _state = TripState.loaded;
285         notifyListeners();
286         return true;
287       },
288     );
289   }
290
291   Future<bool> createTrip(
292       String title, DateTime start, DateTime end, List<String> places)
    async {
293     _state = TripState.loading;
294     _errorMessage = null;
295     notifyListeners();
296
297     final result = await createTripPlanUseCase(title, start, end,
    places);
298
299     return result.fold(
300       (failure) {
301         _state = TripState.error;
302         _errorMessage = failure.message;
303         notifyListeners();
304         return false;
305       },
306       (trip) {
307         _trips.add(trip);
308         _state = TripState.loaded;
309         notifyListeners();
310         return true;
```

```
311       },
312     );
313   }
314
315   Future<void> toggleVisitedStatus(String tripId, int dayIndex, int
      activityIndex) async {
316     if (_currentPlan == null) return;
317
318     final updatedDays = List<TripDay>.from(_currentPlan!.days);
319     final day = updatedDays[dayIndex];
320     final activities = List<ItineraryItem>.from(day.activities);
321     final activity = activities[activityIndex];
322
323     final newVisitedStatus = !activity.isVisited;
324
325     // Optimistic UI update
326     activities[activityIndex] = activity.copyWith(isVisited:
      newVisitedStatus);
327     updatedDays[dayIndex] = TripDay(
328       dayNumber: day.dayNumber,
329       activities: activities,
330       dailySummary: day.dailySummary,
331       transportTips: day.transportTips,
332     );
333
334     _currentPlan = AIItinerary(
335       title: _currentPlan!.title,
336       duration: _currentPlan!.duration,
337       days: updatedDays,
338     );
339     notifyListeners();
340
341     // Call backend if tripPlaceId is available
342     if (activity.tripPlaceId != null) {
343       final parsedTripId = int.tryParse(tripId);
344       if (parsedTripId != null) {
345         await updatePlaceVisitedStatusUseCase(parsedTripId, activity.
      tripPlaceId!, newVisitedStatus);
```

```
346         // Note: we don't rollback on error for now to keep it simple, but we could handle it.
347       }
348     }
349   }
350 }
```

Appendix C

Formal Use Case Specifications

C.1 Overview

To thoroughly document the system's functional capabilities and ensure all business requirements were met, formal Use Case specifications were drafted during the System Analysis phase. This appendix details the primary use cases that govern the interactions between the User (Actor) and the Kemora system.

C.2 Use Case UC-01: Generate AI Itinerary

Use Case ID	UC-01
Name	Generate AI Itinerary
Primary Actor	Authenticated User
Description	The user requests a multi-day travel itinerary generated by AI based on specific constraints (budget, duration, interests).
Pre-conditions	1. User is authenticated with a valid JWT. 2. User has selected a destination Governorate.
Main Flow	1. User navigates to the Trip Planner module. 2. User inputs Duration (1-7 days), Budget (Low/Moderate/High), and Interests. 3. System validates the inputs and displays a loading state. 4. Backend retrieves available places within a 30km radius of the destination. 5. Backend constructs the RAG context prompt and queries OpenRouter. 6. OpenRouter returns a JSON itinerary. 7. Backend parses JSON and fetches missing images concurrently. 8. System renders the interactive day-by-day roadmap.
Alternative Flow	*AI Service Unavailable:* 5a. OpenRouter returns a 500/503 error. 5b. Backend attempts failover to secondary free model. 5c. If all models fail, backend returns a graceful error. System alerts the user to try again later.
Post-conditions	The trip is generated and temporarily stored in the application state.

Table C.1: Use Case: Generate AI Itinerary

C.3 Use Case UC-02: Swap Itinerary Activity

Use Case ID	UC-02
Name	Swap Itinerary Activity
Primary Actor	Authenticated User
Description	The user dislikes a specific place recommended by the AI and requests a contextual alternative.
Pre-conditions	User has successfully generated an AI itinerary (UC-01).
Main Flow	1. User taps "Swap" on a specific activity card in the roadmap. 2. User optionally inputs a reason (e.g., "I want a cheaper restaurant"). 3. System sends the current place and reason to the backend. 4. Backend constructs a targeted replacement prompt. 5. AI returns a JSON object with a single replacement place. 6. System dynamically updates the UI to reflect the new activity.
Post-conditions	The local itinerary state is mutated with the new activity.

Table C.2: Use Case: Swap Itinerary Activity

C.4 Use Case UC-03: Publish Social Post

Use Case ID	UC-03
Name	Publish Social Post
Primary Actor	Authenticated User
Description	The user shares an experience, photo, or linked trip to the community feed.
Pre-conditions	User is authenticated.
Main Flow	1. User navigates to the Social Feed and taps "Create Post". 2. User types content and selects an image from the device gallery. 3. System uploads the image to Cloudinary and receives a secure URL. 4. System submits the post content and image URL to the backend. 5. Backend saves the post to the database and awards Gamification Points to the user. 6. System refreshes the feed to display the new post.
Post-conditions	The post is visible publicly; User's point total is incremented.

Table C.3: Use Case: Publish Social Post

C.5 Use Case UC-04: Offline Fallback (Future Work)

Use Case ID	UC-04
Name	Offline Fallback (Future Work)
Primary Actor	Any User
Description	*Currently not implemented.* In the future, the user will be able to attempt to use the app without an active internet connection and view read-only cached data.
Main Flow	1. (Future) User opens the application. 2. (Future) System detects offline state. 3. (Future) System displays a "No Internet Connection" persistent banner. 4. (Future) Actions requiring the backend (AI generation, posting) are visually disabled.

Table C.4: Use Case: Offline Fallback (Future Work)

C.6 Use Case UC-05: User Registration

Use Case ID	UC-05
Name	User Registration
Primary Actor	Guest User
Description	A new user creates an account to access authenticated features.
Pre-conditions	User is not authenticated.
Main Flow	1. User navigates to the Registration screen. 2. User enters Name, Email, Password, and confirms Password. 3. System validates input formats (e.g., valid email, password strength). 4. System submits credentials to the backend. 5. Backend creates a new user record and generates a JWT. 6. System navigates to the Home screen.
Alternative Flow	*Email Already Exists:* 5a. Backend returns a 409 Conflict error. 5b. System displays an error message prompting the user to login.
Post-conditions	User account is created and user is authenticated.

Table C.5: Use Case: User Registration

C.7 Use Case UC-06: User Login

Use Case ID	UC-06
Name	User Login
Primary Actor	Guest User
Description	An existing user authenticates to access personal data and features.
Pre-conditions	User is not authenticated.
Main Flow	1. User navigates to the Login screen. 2. User enters Email and Password. 3. System submits credentials to the backend. 4. Backend verifies credentials and returns a JWT. 5. System stores JWT securely and navigates to the Home screen.
Alternative Flow	*Invalid Credentials:* 4a. Backend returns a 401 Unauthorized error. 4b. System displays an error message indicating incorrect email or password.
Post-conditions	User is authenticated and can access secured modules.

Table C.6: Use Case: User Login

C.8 Use Case UC-07: Edit Profile

Use Case ID	UC-07
Name	Edit Profile
Primary Actor	Authenticated User
Description	The user updates their profile information such as display name, bio, or avatar.
Pre-conditions	User is authenticated.
Main Flow	1. User navigates to Profile Settings. 2. User edits display name or bio, and optionally selects a new avatar image. 3. If an image is selected, System uploads it to Cloudinary. 4. System submits updated profile details to the backend. 5. Backend updates user record and returns the updated profile. 6. System reflects changes on the user's profile view.
Post-conditions	User's profile is updated in the database.

Table C.7: Use Case: Edit Profile

C.9 Use Case UC-08: Like Post

Use Case ID	UC-08
Name	Like Post
Primary Actor	Authenticated User
Description	The user likes a post on the community feed to show appreciation.
Pre-conditions	User is authenticated and viewing the Social Feed.
Main Flow	1. User taps the "Like" button on a specific post. 2. System immediately toggles the like icon (optimistic UI update). 3. System sends a like request to the backend. 4. Backend registers the like and increments the post's like count.
Alternative Flow	*Network Failure:* 3a. Request to backend fails. 3b. System reverts the optimistic UI update and shows an error snackbar.
Post-conditions	The post's like count is incremented, and the user's interaction is recorded.

Table C.8: Use Case: Like Post

C.10 Use Case UC-09: Comment on Post

Use Case ID	UC-09
Name	Comment on Post
Primary Actor	Authenticated User
Description	The user adds a comment to a social feed post to engage in discussion.
Pre-conditions	User is authenticated.
Main Flow	1. User taps on a post to view its detail or taps the comment icon. 2. User types a comment in the text field and taps "Send". 3. System submits the comment to the backend. 4. Backend saves the comment and associates it with the post. 5. System updates the comment list to display the new comment.
Post-conditions	The comment is publicly visible on the post.

Table C.9: Use Case: Comment on Post

C.11 Use Case UC-10: Delete Post

Use Case ID	UC-10
Name	Delete Post
Primary Actor	Authenticated User
Description	The user deletes a post they previously created.
Pre-conditions	User is authenticated and is the author of the target post.
Main Flow	1. User taps the options menu on their post and selects "Delete". 2. System prompts for confirmation. 3. User confirms deletion. 4. System sends a delete request to the backend. 5. Backend validates ownership and marks the post as deleted (or removes it). 6. System removes the post from the local feed state.
Post-conditions	The post is no longer visible on the social feed.

Table C.10: Use Case: Delete Post

C.12 Use Case UC-11: Redeem Voucher

Use Case ID	UC-11
Name	Redeem Voucher
Primary Actor	Authenticated User
Description	The user spends earned gamification points to redeem a discount voucher.
Pre-conditions	User is authenticated and has sufficient points for the desired voucher.
Main Flow	1. User navigates to the Rewards/Store section. 2. User selects a voucher and taps "Redeem". 3. System prompts for confirmation showing the point cost. 4. User confirms the redemption. 5. Backend deducts the points and generates a unique voucher code. 6. System displays the voucher code and adds it to the user's wallet.
Alternative Flow	*Insufficient Points:* 3a. System detects insufficient points and disables the "Redeem" button, displaying a message.
Post-conditions	User's point balance is decreased, and a new voucher is acquired.

Table C.11: Use Case: Redeem Voucher

C.13 Use Case UC-12: View Badges

Use Case ID	UC-12
Name	View Badges
Primary Actor	Authenticated User
Description	The user views their earned gamification badges and progress toward locked ones.
Pre-conditions	User is authenticated.
Main Flow	1. User navigates to the Gamification Profile. 2. System fetches the user's unlocked badges and progress stats from the backend. 3. System renders a grid of badges, highlighting unlocked ones and graying out locked ones. 4. User taps a specific badge to see its description and unlock criteria.
Post-conditions	No state change.

Table C.12: Use Case: View Badges

C.14 Use Case UC-13: Filter Places List

Use Case ID	UC-13
Name	Filter Places List
Primary Actor	Any User
Description	The user filters the list of places to only show specific types of places (e.g., Ancient Places, Museums, Restaurants).
Pre-conditions	User is on the Places screen.
Main Flow	1. User views the category filter chips at the top of the places list. 2. User selects a specific category chip (e.g., Museums). 3. System updates the places list to display only places matching the selected category.
Post-conditions	The places list is updated to reflect the active filter.

Table C.13: Use Case: Filter Places List

C.15 Use Case UC-14: Save Itinerary

Use Case ID	UC-14
Name	Save Itinerary
Primary Actor	Authenticated User
Description	The user saves an AI-generated itinerary to their profile for future reference.
Pre-conditions	User is authenticated and has generated an itinerary (UC-01).
Main Flow	1. User taps "Save Itinerary" on the roadmap screen. 2. System prompts for an optional custom name for the trip. 3. System submits the itinerary JSON and metadata to the backend. 4. Backend stores the itinerary in the database linked to the user. 5. System displays a success confirmation.
Post-conditions	The itinerary is permanently saved and accessible from the user's profile.

Table C.14: Use Case: Save Itinerary

C.16 Use Case UC-15: Rate and Review Place

Use Case ID	UC-15
Name	Rate and Review Place
Primary Actor	Authenticated User
Description	The user leaves a 1-5 star rating and an optional text review for a specific location.
Pre-conditions	User is authenticated.
Main Flow	1. User navigates to a Place Details screen. 2. User taps "Write a Review". 3. User selects a star rating and types a review. 4. System submits the review data to the backend. 5. Backend saves the review, updates the place's average rating, and awards gamification points. 6. System refreshes the place details to show the new review.
Post-conditions	The place's rating is updated, and the user earns points.

Table C.15: Use Case: Rate and Review Place

C.17 Use Case UC-16: Search Places

Use Case ID	UC-16
Name	Search Places
Primary Actor	Any User
Description	The user searches for a specific location, monument, or restaurant by name.
Pre-conditions	None.
Main Flow	1. User taps the search bar on the Home or Map screen. 2. User types a search query. 3. System debounces the input and sends a search request to the backend. 4. Backend queries the database via full-text search and returns matches. 5. System displays a list of matching places with thumbnails. 6. User taps a result to view Place Details.
Alternative Flow	*No Results Found:* 4a. Backend returns an empty list. 5a. System displays an empty state illustration with "No places found".
Post-conditions	User views the desired place details.

Table C.16: Use Case: Search Places

Appendix D

Database Data Dictionary

D.1 Overview

A robust relational database relies on strict data typing, foreign key constraints, and indexing. This appendix serves as the comprehensive Data Dictionary for the Kemora SQL Server database, detailing the exact schema generated by the Entity Framework Core migrations.

D.2 Table: ApplicationUsers

Stores the core identity and gamification metrics for authenticated users.

Column Name	**Data Type**	**Description / Constraints**
Id	NVARCHAR(450)	Primary Key. Generated UUID (Inherited from IdentityUser).
UserName	NVARCHAR(256)	Nullable. User's chosen username (Inherited).
NormalizedUserName	NVARCHAR(256)	Nullable. Uppercase username for fast lookups (Inherited).
Email	NVARCHAR(256)	Unique Index. User's login email (Inherited).
NormalizedEmail	NVARCHAR(256)	Nullable. Uppercase email (Inherited).
EmailConfirmed	BIT	Required. Indicates if email is verified (Inherited).
PasswordHash	NVARCHAR(MAX)	Nullable. Bcrypt/PBKDF2 hashed password string (Inherited).

SecurityStamp	NVARCHAR(MAX)	Nullable. Value that changes on credential change (Inherited).
ConcurrencyStamp	NVARCHAR(MAX)	Nullable. Concurrency token (Inherited).
PhoneNumber	NVARCHAR(MAX)	Nullable. User's phone number (Inherited).
PhoneNumberConfirmed	BIT	Required. Indicates if phone is verified (Inherited).
TwoFactorEnabled	BIT	Required. Indicates if 2FA is enabled (Inherited).
LockoutEnd	DATETIMEOFFSET	Nullable. When the lockout ends (Inherited).
LockoutEnabled	BIT	Required. Indicates if user can be locked out (Inherited).
AccessFailedCount	INT	Required. Number of failed login attempts (Inherited).
FullName	NVARCHAR(MAX)	Required. User's display name.
ProfilePictureUrl	NVARCHAR(MAX)	Nullable.
Bio	NVARCHAR(MAX)	Nullable.
Country	NVARCHAR(MAX)	Required. User's country of origin.
TotalPoints	INT	Default: 0. Aggregated gamification score.
RefreshToken	NVARCHAR(MAX)	Nullable. Used for issuing new JWTs without re-login.
RefreshTokenExpiryTime	DATETIME2	Required. Timestamp of token expiration.
UserPreferencesJSON	NVARCHAR(MAX)	Nullable. Stores Budget, Vibe, Pace etc.

Table D.1: ApplicationUsers Table Schema

D.3 Table: Badges

Stores gamification badges that users can earn.

Column Name	Data Type	Description / Constraints
BadgeID	INT	Primary Key. Auto-incremented.
Name	NVARCHAR(MAX)	Required.
Description	NVARCHAR(MAX)	Required.
IconUrl	NVARCHAR(MAX)	Required.
Criteria	NVARCHAR(MAX)	Required.
PointsReward	INT	Required.

Table D.2: Badges Table Schema

D.4 Table: UserBadges

Mapping table between users and the badges they have earned.

Column Name	Data Type	Description / Constraints
UserID	NVARCHAR(450)	Composite Key component / Foreign Key → ApplicationUsers(Id).
BadgeID	INT	Composite Key component / Foreign Key → Badges(BadgeID).
EarnedAt	DATETIME2	Required. Timestamp of when the badge was earned.

Table D.3: UserBadges Table Schema

D.5 Table: UserPoints

History of points earned by users.

Column Name	Data Type	Description / Constraints
UserPointID	INT	Primary Key. Auto-incremented.

PointsGained	INT	Required.
GainedAt	DATETIME2	Required.
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).
SourcePlaceID	INT	Nullable. Foreign Key → Places(PlaceID).

Table D.4: UserPoints Table Schema

D.6 Table: UserFavorites

Stores places favorited by users.

Column Name	Data Type	Description / Constraints
UserID	NVARCHAR(450)	Composite Key component / Foreign Key → ApplicationUsers(Id).
PlaceID	INT	Composite Key component / Foreign Key → Places(PlaceID).

Table D.5: UserFavorites Table Schema

D.7 Table: Notifications

Stores notifications for users.

Column Name	Data Type	Description / Constraints
NotificationID	INT	Primary Key. Auto-incremented.
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).
Title	NVARCHAR(MAX)	Required.
Message	NVARCHAR(MAX)	Required.
IsRead	BIT	Required.

CreatedAt	DATETIME2	Required.

Table D.6: Notifications Table Schema

D.8 Table: Governorates

Represents the primary geographical divisions of Egypt.

Column Name	Data Type	Description / Constraints
GovernorateID	INT	Primary Key. Auto-incremented identity.
Name	NVARCHAR(MAX)	Required. Name (e.g., "Cairo", "Alexandria").
Region	NVARCHAR(MAX)	Required. Categorical region (e.g., "Nile Valley", "Red Sea").
ImageURL	NVARCHAR(MAX)	Nullable.
Latitude	DECIMAL(10, 8)	Required.
Longitude	DECIMAL(11, 8)	Required.

Table D.7: Governorates Table Schema

D.9 Table: Categories

Categories of places.

Column Name	Data Type	Description / Constraints
CategoryID	INT	Primary Key. Auto-incremented.
Name	NVARCHAR(MAX)	Required. (e.g., "Natural", "Historical")

Table D.8: Categories Table Schema

D.10 Table: PlaceTypes

Specific types of places under a category.

Column Name	Data Type	Description / Constraints
TypeID	INT	Primary Key. Auto-incremented.
GoogleType	NVARCHAR(MAX)	Required.
DisplayName	NVARCHAR(MAX)	Required.
CategoryID	INT	Required. Foreign Key → Categories(CategoryID).

Table D.9: PlaceTypes Table Schema

D.11 Table: Places

The central repository of all tourist attractions, hotels, and restaurants.

Column Name	Data Type	Description / Constraints
PlaceID	INT	Primary Key. Auto-incremented.
FoursquareId	NVARCHAR(MAX)	Nullable.
Name	NVARCHAR(MAX)	Required.
Description	NVARCHAR(MAX)	Nullable.
Address	NVARCHAR(MAX)	Nullable.
Latitude	DECIMAL(10, 8)	Required.
Longitude	DECIMAL(11, 8)	Required.
Phone	NVARCHAR(MAX)	Nullable.
Website	NVARCHAR(MAX)	Nullable.
Rating	DECIMAL(3, 2)	Required.
PriceLevel	INT	Required.
OpeningHoursJSON	NVARCHAR(MAX)	Nullable.
MainImageURL	NVARCHAR(MAX)	Nullable.
GovernorateID	INT	Nullable. Foreign Key → Governorates(GovernorateID).
PlaceTypeID	INT	Nullable. Foreign Key → PlaceTypes(TypeID).
LastEnrichedAt	DATETIME2	Nullable.

Source	NVARCHAR(MAX)	Nullable.
GoogleDataId	NVARCHAR(100)	Nullable. Maximum length: 100.

Table D.10: Places Table Schema

D.12 Table: Photos

Photos associated with places.

Column Name	Data Type	Description / Constraints
PhotoID	INT	Primary Key. Auto-incremented.
ImageURL	NVARCHAR(MAX)	Required.
IsMain	BIT	Required.
PlaceID	INT	Required. Foreign Key → Places(PlaceID).

Table D.11: Photos Table Schema

D.13 Table: Reviews

Reviews for places.

Column Name	Data Type	Description / Constraints
ReviewID	INT	Primary Key. Auto-incremented.
AuthorName	NVARCHAR(MAX)	Required.
Rating	INT	Required.
Text	NVARCHAR(MAX)	Required.
PlaceID	INT	Required. Foreign Key → Places(PlaceID).

Table D.12: Reviews Table Schema

D.14 Table: Events

Events hosted at places.

Column Name	Data Type	Description / Constraints
EventID	INT	Primary Key. Auto-incremented.
Name	NVARCHAR(MAX)	Required.
StartDate	DATETIME2	Required.
EndDate	DATETIME2	Required.
PlaceID	INT	Required. Foreign Key → Places(PlaceID).

Table D.13: Events Table Schema

D.15 Table: Trips

Stores the metadata for AI-generated or custom-built travel itineraries.

Column Name	Data Type	Description / Constraints
TripID	INT	Primary Key. Auto-incremented.
Name	NVARCHAR(MAX)	Required.
Description	NVARCHAR(MAX)	Required.
StartDate	DATETIME2	Required.
EndDate	DATETIME2	Required.
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).

Table D.14: Trips Table Schema

D.16 Table: TripPlaces

The intersection table representing the sequence of places within a Trip.

Column Name	Data Type	Description / Constraints
TripPlaceID	INT	Primary Key. Auto-incremented.
TripID	INT	Required. Foreign Key → Trips(TripID).

PlaceID	INT	Required. Foreign Key → Places(PlaceID).
VisitDate	DATETIME2	Required.
Notes	NVARCHAR(MAX)	Nullable.
IsVisited	BIT	Required.

Table D.15: TripPlaces Table Schema

D.17 Table: PrecomputedTripPlans

Cached AI-generated trip plans.

Column Name	**Data Type**	**Description / Constraints**
Id	INT	Primary Key. Auto-incremented.
CacheKey	NVARCHAR(255)	Required. Maximum length: 255.
ItineraryJson	NVARCHAR(MAX)	Required.
PlacesJson	NVARCHAR(MAX)	Required.
CreatedAt	DATETIME2	Required.

Table D.16: PrecomputedTripPlans Table Schema

D.18 Table: Posts

The core entity for the social community feed.

Column Name	**Data Type**	**Description / Constraints**
PostID	INT	Primary Key. Auto-incremented.
Content	NVARCHAR(MAX)	Required. The text payload of the post.
CreatedAt	DATETIME2	Required. Timestamp.
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).

LinkedTripId	INT	Nullable. Foreign Key → Trips(TripID).
LocationId	INT	Nullable. Foreign Key → Places(PlaceID).

Table D.17: Posts Table Schema

D.19 Table: Stories

User stories.

Column Name	Data Type	Description / Constraints
StoryID	INT	Primary Key. Auto-incremented.
MediaUrl	NVARCHAR(MAX)	Required.
MediaType	NVARCHAR(MAX)	Required.
CreatedAt	DATETIME2	Required.
ExpiresAt	DATETIME2	Required.
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).
LocationId	INT	Nullable. Foreign Key → Places(PlaceID).

Table D.18: Stories Table Schema

D.20 Table: PostMedia

Media attachments for posts.

Column Name	Data Type	Description / Constraints
MediaID	INT	Primary Key. Auto-incremented.
MediaURL	NVARCHAR(MAX)	Required.
MediaType	NVARCHAR(MAX)	Required. ("Image", "Video")

PostID	INT	Required. Foreign Key → Posts(PostID).

Table D.19: PostMedia Table Schema

D.21 Table: PostReactions

Reactions to posts.

Column Name	Data Type	Description / Constraints
ReactionType	NVARCHAR(MAX)	Required. ("Like", "Love")
ReactedAt	DATETIME2	Required.
PostID	INT	Composite Key component / Foreign Key → Posts(PostID).
UserID	NVARCHAR(450)	Composite Key component / Foreign Key → ApplicationUsers(Id).

Table D.20: PostReactions Table Schema

D.22 Table: Comments

Comments on posts.

Column Name	Data Type	Description / Constraints
CommentID	INT	Primary Key. Auto-incremented.
Content	NVARCHAR(MAX)	Required.
CreatedAt	DATETIME2	Required.
PostID	INT	Required. Foreign Key → Posts(PostID).
UserID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).

ParentCommentId	INT	Nullable. Foreign Key → Comments(CommentID).

Table D.21: Comments Table Schema

D.23 Table: CommentMedia

Media attachments for comments.

Column Name	Data Type	Description / Constraints
MediaID	INT	Primary Key. Auto-incremented.
MediaURL	NVARCHAR(MAX)	Required.
MediaType	NVARCHAR(MAX)	Required.
CommentID	INT	Required. Foreign Key → Comments(CommentID).

Table D.22: CommentMedia Table Schema

D.24 Table: CommentReactions

Reactions to comments.

Column Name	Data Type	Description / Constraints
ReactionType	NVARCHAR(MAX)	Required.
ReactedAt	DATETIME2	Required.
CommentID	INT	Composite Key component / Foreign Key → Comments(CommentID).
UserID	NVARCHAR(450)	Composite Key component / Foreign Key → ApplicationUsers(Id).

Table D.23: CommentReactions Table Schema

D.25 Table: Messages

Direct messages between users.

Column Name	Data Type	Description / Constraints
MessageID	INT	Primary Key. Auto-incremented.
Content	NVARCHAR(MAX)	Required.
SentAt	DATETIME2	Required.
IsRead	BIT	Required.
SenderID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).
ReceiverID	NVARCHAR(450)	Required. Foreign Key → ApplicationUsers(Id).

Table D.24: Messages Table Schema

Appendix E

Flutter UI Widget Dictionary

This appendix provides a comprehensive reference to the primary Flutter UI widgets and the design system implemented for the Kemora application. The frontend architecture relies on a highly modular and reusable component strategy, leveraging the "Modern Heritage" design aesthetic.

E.1 Modern Heritage Design System

The "Modern Heritage" design system is implemented in Flutter using Material 3's `ThemeData` and custom token classes (`AppColors`, `AppTypography`, and `AppShadows`). It combines rich, earthy tones like `primaryContainer` (#c5a021, Primary Gold) and `secondary` (#435b9f, Primary Blue) with modern, clean typography and subtle drop shadows for an elegant, premium feel.

- **Color Scheme:** The `AppColors` class defines all color tokens, bridging semantic naming (e.g., `primary`, `surfaceContainerHighest`) with fixed brand colors.
- **Typography:** `AppTypography` extends Material 3's `TextTheme`, applying brand-specific font weights and letter spacing (tracking) to establish hierarchy.
- **Theme Injection:** `AppTheme.lightTheme` configures the global `ThemeData`, overriding default styles for `AppBar`, `ElevatedButton`, `Card`, and `InputDecoration` to ensure consistency without redundant inline styling.

E.2 Core Reusable Widgets

The `lib/presentation/widgets` directory contains all globally shared UI components. These components encapsulate complex layout, state, and animation logic.

E.2.1 EditorialPlaceCard

The `EditorialPlaceCard` is a visually striking component used to display points of interest, accommodations, and tours. It features a large cover image, a soft gradient overlay for text legibility, and an integrated `GlassmorphismContainer` for the favorite action.

```
@override
Widget build(BuildContext context) {
  return GestureDetector(
    onTap: onTap,
    child: AspectRatio(
      aspectRatio: aspectRatio > 1 ? aspectRatio : 1 / (1 /
 aspectRatio),
      child: Container(
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(24),
          boxShadow: AppShadows.ambient,
        ),
        child: ClipRRect(
          borderRadius: BorderRadius.circular(24),
          child: Stack(
            fit: StackFit.expand,
            children: [
              // Image or Placeholder
              // ... inline image loading logic ...

              // Gradient Overlay (Soft Lapis-to-transparent)
              Container(
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    begin: Alignment.bottomCenter,
                    end: Alignment.topCenter,
                    colors: [
                      AppColors.secondary.withOpacity(0.9),
                      AppColors.secondary.withOpacity(0.4),
                      Colors.transparent,
                    ],
                    stops: const [0.0, 0.4, 0.8],
```

```
                    ),
                  ),
                ),

                // Content Overlay ...
                // ... inline widget composition ...
              ],
            ),
          ),
        ),
      ),
    );
  }
```

Listing E.1: EditorialPlaceCard Build Method Excerpt

E.2.2 Itinerary Widgets

The itinerary timeline is composed of several modular widgets defined in `itinerary_widgets.dart`, including `DayHeader`, `ActivityTimelineTile`, and `PremiumActivityCard`.

The `ActivityTimelineTile` constructs the vertical timeline layout, seamlessly connecting adjacent activities using custom-drawn lines and circular nodes.

```
class ActivityTimelineTile extends StatelessWidget {
  final ItineraryItem activity;
  final bool isLast;
  const ActivityTimelineTile({
    super.key,
    required this.activity,
    this.isLast = false,
  });

  @override
  Widget build(BuildContext context) {
    return IntrinsicHeight(
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Timeline logic
```

```
        Column(
          children: [
            Container(
              width: 16,
              height: 16,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color: AppColors.primaryContainer,
                border: Border.all(color: Colors.white, width: 3),
                boxShadow: [
                  BoxShadow(
                    color: AppColors.primaryContainer.withValues(
alpha: 0.3),
                    blurRadius: 4,
                    offset: const Offset(0, 2),
                  ),
                ],
              ),
            ),
            if (!isLast)
              Expanded(
                child: Container(
                  width: 2,
                  color: AppColors.primaryContainer,
                  margin: const EdgeInsets.symmetric(vertical: 4),
                ),
              ),
          ],
        ),
        const SizedBox(width: 16),
        // Content Card
        Expanded(
          child: Padding(
            padding: const EdgeInsets.only(bottom: 24),
            child: PremiumActivityCard(activity: activity),
          ),
        ),
      ],
    ),
```

```
    );
  }
}
```

Listing E.2: ActivityTimelineTile Layout Structure

E.2.3 FloatingNavBar

The `FloatingNavBar` is an animated, glassmorphic bottom navigation bar. It utilizes `FadeSlideIn` to animate onto the screen and `BackdropFilter` for a blurred background effect.

E.2.4 KemoraAppBar

A globally used `PreferredSizeWidget` that implements a translucent, frosted-glass header with the `KEMORA` brand text using wide tracking (letter spacing).

E.3 Animation and Visual Effect Utilities

Kemora incorporates subtle micro-interactions to enhance the user experience.

E.3.1 GlassmorphismContainer

A reusable wrapper that applies a blur effect (`ImageFilter.blur`) to its background, providing a frosted glass appearance.

```
Container(
  decoration: BoxDecoration(
    color: color.withValues(alpha: opacity),
    borderRadius: effectiveRadius,
    border: border,
  ),
  child: ClipRRect(
    borderRadius: effectiveRadius,
    child: BackdropFilter(
      filter: ImageFilter.blur(sigmaX: blurRadius, sigmaY: blurRadius),
      child: Padding(
        padding: padding ?? EdgeInsets.zero,
```

```
          child: child,
        ),
      ),
    ),
)
```

Listing E.3: GlassmorphismContainer BackdropFilter

E.3.2 FadeSlideIn & TapScale

- `FadeSlideIn`: A `StatefulWidget` utilizing an `AnimationController` and `CurvedAnimation` to smoothly fade and slide children into view when the widget mounts.
- `TapScale`: A wrapper that listens to `TapDown` and `TapUp` events to slightly scale down (e.g., `0.96`) the wrapped widget while pressed, creating a tactile button response.

Glossary and Abbreviations

API Application Programming Interface. A set of rules that allows different software entities to communicate.

DTO Data Transfer Object. An object that carries data between processes to reduce the number of method calls and network overhead.

EF Core Entity Framework Core. An open-source, lightweight, and extensible object-relational mapper (O/RM) for the .NET ecosystem.

JWT JSON Web Token. An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.

LLM Large Language Model. A deep learning algorithm, utilizing transformer architecture, that can recognize, summarize, translate, predict, and generate human-like text.

POI Point of Interest. A specific geographical location (e.g., restaurant, museum, or landmark) that a user may find useful or interesting.

RAG Retrieval-Augmented Generation. An AI architectural framework that retrieves data from external knowledge bases and injects it into an LLM's context window to ground the generation and prevent hallucinations.

SERP Search Engine Results Page. The page displayed by a search engine in response to a query, previously utilized for targeted image fetching.

UI/UX User Interface / User Experience. The graphical layout and the holistic interactive experience provided to the end-user by the application.

UUID Universally Unique Identifier. A 128-bit label used for information in computer systems, heavily utilized as Primary Keys in the Kemora database.